Resumen

Resumen

Palabras clave: interfaz cerebro-computador, actividad neuronal intracortical, decodificación neuronal, OWSM, aprendizaje por transferencia, CTC

Abstract

Abstract

Key words: brain–computer interface, intracortical neural activity, neural decoding, OWSM, transfer learning, CTC

Índice general

Índice general III
Índice de figuras VII
Índice de tablas VII

1 Introducción 1
1.1 Introducción . . . 1
1.1.1 Contexto . . . 1
1.1.2 Del cuerpo al cerebro: evolución de los sistemas de comunicación . . . 1
1.2 Motivación . . . 3
1.2.1 Motivación . . . 3
2 Planteamiento del trabajo 5
2.1 Planteamiento del trabajo . . . 5
2.2 Objetivos . . . 6
2.2.1 Objetivos específicos . . . 6
2.3 Alcance y limitaciones . . . 6
2.4 Estructura de la memoria . . . 7
3 Estado del arte 9
3.1 Modalidades de registro en interfaces cerebro-computador . . . 9
3.2 Evolución de las neuroprótesis del habla . . . 10
3.3 Anatomía de un decodificador *brain-to-text* . . . 11
3.3.1 El cambio entre días . . . 11
3.3.2 El codificador . . . 11
3.3.3 La salida y el modelo de lenguaje . . . 11
3.4 El *benchmark* Brain-to-Text '25 . . . 12
3.5 Modelos de habla preentrenados y transferencia a la señal neuronal . . . 12
3.6 Comparativa de los sistemas revisados . . . 12
3.7 Posicionamiento de este trabajo . . . 13
4 Conjunto de datos 15
4.1 Selección del conjunto de datos . . . 15
4.2 El conjunto Brain-to-Text '25 . . . 16
4.2.1 Origen y participante . . . 16
4.2.2 Adquisición de la señal . . . 16
4.2.3 Organización de los datos . . . 17
4.2.4 Particiones y composición . . . 18
4.3 Análisis exploratorio . . . 18
5 Marco teórico 21
5.1 Planteamiento del problema . . . 21
5.2 Datos de entrada al modelo . . . 21
5.2.1 Lotes, relleno y máscaras . . . 21
5.2.2 Capa de entrada por sesión . . . 22
5.3 *Frontend* . . . 22
5.4 OWSM . . . 24

5.4.1 Origen y versiones . . . 24
5.4.2 Arquitectura . . . 24
5.4.3 Atención . . . 25
5.4.4 Codificador E-Branchformer . . . 25
5.4.5 Decodificador Transformer . . . 26
5.5 Clasificación temporal conexionista (CTC) . . . 27
5.5.1 Formulación y cabeza CTC . . . 27
5.5.2 Pérdida conjunta e InterCTC . . . 28
5.5.3 Decodificación . . . 28
5.6 Adaptación de un modelo preentrenado . . . 28
5.6.1 LoRA . . . 29
5.7 Entrenamiento . . . 29
5.8 Métricas de evaluación . . . 30
6 Metodología **33**
6.1 Adaptación de OWSM a la señal neuronal . . . 33
6.2 Evaluación . . . 33
6.3 Fase 1: búsqueda del mejor modelo . . . 34
6.3.1 Puesta en marcha . . . 34
6.3.2 Receta general y capa por sesión . . . 35
6.3.3 Codificador (OE2) . . . 35
6.3.4 Decodificador (OE3) . . . 35
6.3.5 *Frontend* (OE1) . . . 35
6.4 Fase 2: comportamiento en el tiempo y cantidad de datos . . . 36
6.4.1 Variabilidad entre sesiones (OE4) . . . 36
6.4.2 Entrenamiento cronológico (OE4) . . . 36
6.4.3 Análisis por palabras . . . 37
7 Resultados y discusión **39**
7.1 Organización de la presentación . . . 39
7.2 Bloques A y H: la aportación del preentrenamiento . . . 40
7.2.1 Resultados . . . 40
7.2.2 Las dos objeciones previsibles, resueltas . . . 40
7.2.3 El control que cierra el argumento . . . 41
7.2.4 La tasa de aprendizaje diferenciada es un requisito de estabilidad . . . 41
7.2.5 Contraste con la literatura previa . . . 41
7.3 Bloque D: la curva de conservación . . . 42
7.4 Bloque F: el tamaño de la arquitectura . . . 43
7.5 Bloque B: granularidad de la unidad de salida . . . 43
7.5.1 Resultados . . . 43
7.5.2 Por qué falló la predicción . . . 44
7.5.3 Matices y limitaciones del hallazgo . . . 44
7.5.4 Respuesta a PI2 . . . 45
7.6 Bloque E: generalización a sesiones no vistas . . . 45
7.7 Bloque C: capacidad del módulo adaptador . . . 46
7.8 Decodificación con modelo de lenguaje . . . 46
7.9 Comparación con el estado del arte . . . 47
7.10 Discusión . . . 47
7.10.1 Síntesis por pregunta de investigación . . . 47
7.10.2 Una lectura conjunta . . . 48
7.10.3 Amenazas a la validez . . . 48
8 Limitaciones y trabajo futuro **49**
8.1 Limitaciones del estudio . . . 49

8.1.1 Alcance de la evidencia . . . 49
8.1.2 Restricciones del entorno de ejecución . . . 49
8.1.3 Elementos del plan que no se completaron . . . 50
8.2 Líneas de trabajo futuro . . . 50
8.2.1 Discriminar qué tipo de preentrenamiento transfiere . . . 50
8.2.2 Atacar la no estacionariedad como problema central . . . 50
8.2.3 Cerrar el eje de granularidad . . . 51
8.2.4 Explotar la eficiencia del codificador reducido . . . 51
8.2.5 Reproducir el hallazgo en otros participantes . . . 51

9 Conclusiones **53**
9.1 Respuesta a las preguntas de investigación . . . 53
9.1.1 PI1: el preentrenamiento de OWSM no transfiere . . . 53
9.1.2 PI2: la unidad óptima es la más lingüística, no la más articulatoria 53
9.1.3 PI3: el conocimiento lingüístico reside fuera del codificador . . . 54
9.2 Aportaciones . . . 54
9.3 Lecciones metodológicas . . . 55
9.4 Valoración final . . . 55

Bibliografía **57**

Apéndices

A Código del pipeline de entrenamiento **63**

B Análisis exploratorio adicional **65**

C Relación del trabajo con los Objetivos de Desarrollo Sostenible de la Agenda 2030 **67**

Índice de figuras

4.1 Esquema de una interfaz cerebro-texto con microelectrodos 17
4.2 Número de palabras por frase 19
4.3 Repetición de frases y solape entre particiones 19
4.4 Frecuencia de las palabras en el entrenamiento 20
4.5 Número de frases por sesión 20

7.1 Evolución de la tasa de error por carácter según la inicialización del codificador. 40

Índice de tablas

3.1 Comparación de las modalidades de registro usadas en BCI del habla. . . 10
3.2 Resumen de los sistemas revisados en este capítulo. 13

4.1 Comparación de los conjuntos de datos considerados. 16
4.2 Composición de Brain-to-Text '25 por corpus de origen 18

6.1 *Frontends* comparados en la fase 1. 36

7.1 Aportación del preentrenamiento. Tasa de error por carácter mínima sobre validación, época del mínimo, presupuesto y coste en unidades de cómputo. 40
7.2 Rendimiento en función de la cantidad de codificador preentrenado que se adapta. La pendiente final es la del descenso de la tasa de error en la última época del presupuesto. 42
7.3 Efecto de la profundidad del codificador con inicialización aleatoria. . . . 43
7.4 Comparación de unidades de salida con decodificación voraz. La columna de error por palabra es el único eje comparable entre filas. 43
7.5 Rendimiento sobre las cinco sesiones excluidas del entrenamiento, según el tratamiento de la capa de sesión. 45
7.6 Comparación de variantes de adaptador. Las tres comparten factor de submuestreo dos. 46
7.7 Resultados preliminares de decodificación con modelo de lenguaje de n-gramas por fusión superficial. 46
7.8 Situación del sistema propuesto frente a la referencia comparable sobre el mismo conjunto de datos. 47

C.1 Grado de relación del trabajo con los Objetivos de Desarrollo Sostenible. . 67

CAPÍTULO 1
Introducción

1.1 Introducción

1.1.1. Contexto

La comunicación oral es una capacidad básica para vivir en sociedad. Sin embargo, hay enfermedades que pueden quitarla casi por completo, aunque la persona siga pensando y entendiendo con normalidad. Y puede ser la causa de muchos trastornos. Se suelen dividir en disartria y . La disartria es un trastorno motor del habla en el que se pierde el control de los músculos del tracto vocal, cuando la incapacidad de producir un habla que se entienda [1] es total, se le llama anartria. Puede aparecer con la esclerosis lateral amiotrófica (ELA), el ictus, el Parkinson, la parálisis cerebral, los traumatismos craneoencefálicos o algunos tumores cerebrales, y su gravedad es variable entre las personas [2]. En la afasia falla el propio lenguaje. En la disartria, en cambio, las zonas del cerebro que planifican el habla siguen funcionando: la persona sabe lo que quiere decir e intenta decirlo, pero la orden no llega bien a los músculos [1].

La ELA y el ictus son dos ejemplos claros de pérdida completa del habla, y también las enfermedades de la mayoría de las personas que participan en los estudios de la siguiente sección. La ELA va destruyendo las neuronas motoras de la corteza, el tronco del encéfalo y la médula espinal, y los músculos se van paralizando, también los que usamos para hablar, tragar y respirar [3]. Su prevalencia en el mundo es de 4,42 casos por cada 100.000 habitantes [4]. En España la tienen entre 4.000 y 4.500 personas y se diagnostican unos tres casos nuevos al día [5]. Entre el 80 % y el 95 % de los pacientes llegan a un punto en el que ya no se pueden comunicar hablando en su día a día [3]. El ictus es mucho más frecuente: en 2021 hubo 11,9 millones de casos nuevos en el mundo [6], en España hay unos 90.000 al año [7], y entre el 20 % y el 42 % de los supervivientes tienen disartria [8]. Cuando daña la parte ventral de la protuberancia, en el tronco del encéfalo, puede provocar el síndrome de cautiverio (*locked-in syndrome*). La persona queda consciente pero con tetraplejia y anartria, y en la forma clásica solo puede comunicarse parpadeando o moviendo los ojos en vertical [9], [10].

Aunque aquí se usen estos dos casos como ejemplo, lo que se busca en este trabajo puede servir a cualquier persona que conserve la intención de hablar y no pueda articularla del todo, sea cual sea la causa.

1.1.2. Del cuerpo al cerebro: evolución de los sistemas de comunicación

Desde hace décadas se han ido creando sistemas para que estas personas puedan comunicarse de otra forma. Los primeros eran tableros con letras o dibujos que la persona

señalaba, y máquinas electromecánicas de escribir que se manejaban con un pulsador o con algún otro movimiento pequeño. En los años sesenta y setenta estas ideas se juntaron y dieron lugar a lo que hoy se llama comunicación aumentativa y alternativa (CAA) [11]. Después llegaron los ordenadores y los sintetizadores de voz, con los que se escribe un mensaje y el equipo lo lee en voz alta. Son sistemas sencillos, no necesitan cirugía y muchos pacientes los usan hasta el final de la enfermedad [3]. Su problema es que son lentos y cansados, y que dependen de que quede algún movimiento voluntario con el que manejarlos [1].

Uno de los sistemas más usados en la ELA es el seguimiento ocular. Una cámara detecta hacia dónde mira la persona y así puede elegir letras o palabras en una pantalla. Se usa mucho en fases avanzadas, cuando ya no se puede hablar ni mover las manos, aunque solo permite una comunicación muy lenta o limitada [12] y en algunos sitios, como al aire libre, estos sistemas no funcionan [13]. Por último, cuando la enfermedad llega a los músculos de los ojos, dejan de servir [12].

Una evolución que se investiga dependiendo de los usuarios son los electrodos de electromiografía (EMG). Se colocan en la cara y el cuello y se puede reconocer lo que una persona intenta decir aunque no salga ningún sonido, lo que se conoce como habla silenciosa [14]. Es un método no invasivo y está más cerca del habla real que ir eligiendo letras. Pero cuanta más severa es la parálisis, menos precisión tienen, hasta el punto de que puede no haber ninguna señal.

Si no queda ningún músculo útil, solo se puede recurrir al propio cerebro. Las interfaces cerebro-computador (*Brain-Computer Interfaces*, BCI) registran su actividad y la convierten en órdenes para un dispositivo. Las primeras usaban electroencefalografía (EEG), con electrodos sobre el cuero cabelludo. En 1988, Farwell y Donchin presentaron un deletreador basado en la onda P300. En una pantalla se iluminan filas y columnas de letras y el sistema detecta la respuesta del cerebro cuando se ilumina la letra que el usuario quiere [15]. Ya en 1999, Birbaumer y su equipo consiguieron que pacientes totalmente paralizados por ELA escribieran mensajes controlando los potenciales corticales lentos de su EEG [16]. El EEG no necesita cirugía, pero la señal llega muy atenuada a través del cráneo y tiene mucho ruido, así que estos sistemas eran muy lentos, de unas pocas letras por minuto.

Para tener mejor señal hay que acercarse a las neuronas, y eso ya obliga a operar. La electrocorticografía (ECoG) coloca electrodos sobre la superficie de la corteza, por debajo del cráneo. En 2016 se publicó el caso de una paciente con ELA en estado de cautiverio que usaba en su casa una BCI de este tipo totalmente implantada. Al intentar mover la mano generaba un «clic» con el que elegía letras y, aunque era lento, le permitía comunicarse cuando su sistema ocular fallaba [13]. Además, esto permitió dar el salto de elegir letras a decodificar palabras directamente a partir del intento de hablar, primero con un vocabulario pequeño [17] y después con frases completas, voz sintetizada y un avatar [18]. La estereoelectroencefalografía (sEEG) usa en cambio electrodos de profundidad, que se implantan a pacientes con epilepsia para localizar el origen de sus crisis. Con estos registros se ha llegado a sintetizar voz en tiempo real a partir del habla imaginada, aunque el resultado todavía no se entendía [19]. Su ventaja es que aprovecha una operación que el paciente ya necesita, pero los electrodos se colocan donde dicta el criterio médico y solo se quedan durante un tiempo corto.

El último paso son las matrices de microelectrodos intracorticales, que penetran en la corteza y registran la actividad de pequeños grupos de neuronas. En 2006 el ensayo BrainGate mostró que un hombre con tetraplejia podía mover un cursor, abrir el correo o abrir y cerrar una mano protésica solo con la actividad de su corteza motora [20]. A partir de ahí se pasó a escribir moviendo un cursor sobre un teclado en pantalla [21], luego a convertir en texto la escritura a mano imaginada [22] y, por último, a decodificar

directamente el habla intentada [23]. Hoy ya hay un participante con ELA que lleva meses usando un sistema así para conversar en su día a día [24]. Con el mismo tipo de implante también se ha conseguido sintetizar la voz al instante, con entonación e incluso cantando melodías sencillas [25]. Son, con diferencia, los sistemas con mejores resultados. A cambio, exigen una operación cerebral y de momento solo los han probado unas pocas personas dentro de ensayos clínicos.

Si se mira toda esta evolución, se ve que cada paso ha aparecido porque el anterior dependía de algo que la enfermedad podía quitar: las manos, los músculos de la boca y los ojos. La actividad del cerebro es lo último que queda, y cuanto más cerca de las neuronas se registra, mejor es la señal pero más invasivo es el método. Las tecnologías BCI han tenido un auge en las últimas décadas y han pasado de controlar cursores o movimientos a poder decodificar lenguaje. Este trabajo se sitúa en esta última etapa, la de los sistemas *brain-to-text*, que traducen a texto la actividad de la corteza motora mientras la persona intenta hablar.

1.2 Motivación

1.2.1. Motivación

La motivación principal de este trabajo es social. Según la encuesta de discapacidad del INE de 2020, en España había unas 947.500 personas de seis años o más con una discapacidad relacionada con la comunicación [26]. No todas tienen las enfermedades descritas en la sección 1.1.1, pero la cifra da una idea de cuánta gente tiene problemas para hacerse entender. Y es un problema que está creciendo, ya que la Sociedad Española de Neurología prevé que los casos de ELA en Europa aumenten un 40 % en los próximos 25 años [5]. Quien no puede hablar depende de otros hasta para pedir lo más básico y, con el tiempo, se va quedando aislado [18].

La comunicación pesa mucho en la calidad de vida. En personas con ELA, contar con un sistema de comunicación desde las primeras fases tiene un efecto positivo en su bienestar y en el de sus cuidadores [27]. En España este problema empieza a tener más reconocimiento, por ejemplo con la Ley 3/2024, conocida como «Ley ELA», que busca mejorar la calidad de vida de estas personas y agilizar el reconocimiento de su discapacidad y su dependencia [28]. Sin embargo, aún no existe una verdadera solución al problema, y seguir investigando puede ayudar a muchas personas que lo necesitan.

Asimismo, me motiva desde el punto de vista técnico, porque las BCI del habla son un campo que avanza muy rápido. En unos veinte años se ha pasado de mover un cursor a decodificar frases completas y sintetizar voz en tiempo real. Estos sistemas ya superan la velocidad de las herramientas de comunicación asistida que existen hoy [1]. Cada poco tiempo se aplican a problemas nuevos [25], y se están consolidando como una de las mejores soluciones para la comunicación. Además, retos públicos como Brain-to-Text han puesto estos datos al alcance de cualquier grupo de investigación, aunque no tenga acceso a pacientes [29], [30]. El principal inconveniente sigue siendo que hay que operar el cerebro. Aun así, ya se están probando opciones menos invasivas, como electrodos que se introducen por un vaso sanguíneo sin abrir el cráneo [31], y en el futuro se espera que estas soluciones se apliquen cada vez más.

Por último, también hay una motivación personal. Siempre me ha interesado la biomedicina, y en especial cómo la tecnología puede ayudar a las personas con discapacidad. He participado en hackathons y, sobre todo, he formado parte durante tres años de Daidalonic, un grupo de estudiantes de la UPV que diseña prótesis y otros proyectos biomédicos. Allí trabajé con la EMG y pude hablar con personas con discapacidad. Pero además pude

ver de primera mano en el Cybathlon 2024 a muchos grupos de investigación utilizando las BCI para ayudar a personas completamente paralizadas, ya sea con la movilidad, el entretenimiento o la comunicación, y supe que quería investigar algo relacionado.

CAPÍTULO 2
Planteamiento del trabajo

2.1 Planteamiento del trabajo

Este trabajo estudia la viabilidad de reutilizar un modelo de reconocimiento del habla preentrenado a gran escala como codificador de señal neuronal en una tarea *brain-to-text*, es decir, para pasar a texto la actividad del cerebro mientras una persona intenta hablar. En concreto, se adapta OWSM (*Open Whisper-style Speech Model*) [32], [33], de la librería ESPnet [34], [35]. OWSM sigue la idea de Whisper [36], pero se entrena solo con datos públicos y sus autores publican el código, los modelos y los registros de entrenamiento [33]. Por eso es una de las mejores opciones abiertas y reproducibles. La idea de fondo es aprovechar lo que estos modelos han aprendido de grandes cantidades de audio en un problema en el que los datos son muy escasos, porque cada grabación necesita a un paciente con un implante y muchas horas de entrenamiento.

La pregunta principal es qué hay que cambiar del modelo para que funcione con señal neuronal y si esos cambios hacen que la idea sea viable. La entrada de OWSM está pensada para audio, así que hay que sustituirla por un módulo nuevo, el *frontend*, que adapte la señal neuronal al resto del modelo. A partir de ahí se analiza qué partes se pueden aprovechar tal cual, cuáles hay que reentrenar y con qué nivel de ajuste. Este proyecto no se centra en la limpieza y el análisis de las señales neuronales, para las que se usa el preprocesado estándar del conjunto de datos, sino en la capacidad del modelo y sus resultados.

Además de la viabilidad, se quiere ver cómo se comporta el modelo en condiciones parecidas a las de un uso real. Por un lado, con palabras y frases que no ha visto durante el entrenamiento. Por otro, con el paso del tiempo. La actividad que registran los electrodos cambia de un día para otro, y para mantener el rendimiento estos sistemas suelen necesitar recalibraciones frecuentes, que obligan al usuario a dejar de usarlos y grabar datos nuevos [37]. Por eso se estudia cuántos datos hacen falta para tener un modelo que no necesite más, y qué se puede concluir sobre la necesidad de datos de este tipo de sistemas.

Tampoco se pretende superar a los mejores sistemas ni competir en los *benchmarks*. Los primeros puestos de estos retos combinan las salidas de varios decodificadores independientes mediante un modelo de lenguaje de gran tamaño [29]. Aquí el interés está en ver hasta dónde llega un modelo de habla abierto por sí solo y entender qué estrategias son las más viables con recursos limitados.

Así pues, el trabajo se divide en dos bloques de experimentos. En el primero se entrena con todas las sesiones y se busca la mejor forma de adaptar OWSM, probando distintas opciones para el *frontend*, el *encoder* y el *decoder*. En el segundo se toma el mejor modelo del primer bloque y se simula su uso real: se entrena solo con las primeras sesiones, en

orden de fecha, y se evalúa tanto en esas sesiones como en las que vienen después. Así se analiza su capacidad predictiva en cada sesión para poder concluir el grado de eficiencia.

2.2 Objetivos

El objetivo general de este trabajo es evaluar la viabilidad de reutilizar un modelo de reconocimiento del habla preentrenado a gran escala, OWSM, para decodificar a texto la actividad neuronal intracortical durante el habla intentada. Para ello se determina qué partes del modelo pueden aprovecharse y cuáles deben adaptarse o reentrenarse, y con qué coste. También se analiza cómo se comporta el sistema ante frases no vistas y con el paso del tiempo, con el fin de estimar cuántos datos necesita.

2.2.1. Objetivos específicos

El objetivo general se concreta en cuatro objetivos específicos. Los tres primeros corresponden al primer bloque de experimentos y el cuarto, al segundo. Las partes del sistema que se nombran en ellos se explican en el capítulo 5.

O1. ***Frontend.*** Diseñar y comparar varios módulos de entrada que adapten la señal neuronal al *encoder* de OWSM, desde una capa lineal sencilla hasta una red convolucional profunda, y con distinto submuestreo temporal. Se mide tanto el resultado como el coste de entrenamiento, para elegir el mejor compromiso entre ambos.

O2. ***Encoder.*** Comprobar si los pesos preentrenados del *encoder* de OWSM aportan algo frente al mismo *encoder* entrenado desde cero cuando la entrada es actividad neuronal en lugar de audio.

O3. ***Decoder.*** Evaluar si el *decoder* preentrenado de OWSM, que aporta el conocimiento del lenguaje, puede usarse sin modificar, si mejora con una adaptación ligera como LoRA [38] o si es mejor entrenarlo desde cero.

O4. **Ventana temporal.** Medir cómo varía el error del mejor modelo de una sesión a otra y simular después un uso real del sistema, entrenando con un número creciente de sesiones en orden cronológico y evaluando en sesiones ya vistas y en sesiones futuras. Con ello se quiere saber cuánto empeora el modelo con el paso del tiempo y cuántos datos hacen falta para que deje de necesitar más. También se analiza qué palabras reconoce mejor y peor, según lo frecuentes que sean en el entrenamiento y si aparecen o no en él, y cómo se comporta con frases que no ha visto.

2.3 Alcance y limitaciones

Este trabajo es un estudio intrapersonal. Se centra en una sola persona, que además habla en inglés, y busca el mejor sistema para ella, no un modelo que funcione con cualquiera. Por eso sus conclusiones no se pueden trasladar sin más a otros participantes. Es lo habitual en este campo, donde la mayoría de los estudios trabajan con un único participante [23], [24]. Además, todo se evalúa en diferido, sobre grabaciones ya hechas y sin estudiar el funcionamiento en tiempo real, y solo se trabaja con habla intentada, en la que la persona intenta hablar aunque no pueda. El habla imaginada queda fuera.

La limitación más importante es el coste. Los sistemas *brain-to-text* actuales son grandes y caros de entrenar y de usar. Además de combinar varios modelos, el decodificador de

referencia de estos retos necesita unos 320 GB de memoria RAM, algo que limita el acceso tanto a los pacientes como a los investigadores [39]. Este trabajo, en cambio, se ha hecho con una sola GPU en Google Colab y con un tiempo y un presupuesto limitados. Cada entrenamiento completo tarda varias horas, así que cada configuración se ha entrenado una sola vez y el número de pruebas ha sido limitado. Las limitaciones que salen de los propios resultados se comentan en el capítulo 8.

2.4 Estructura de la memoria

El resto de la memoria se organiza de la siguiente forma. El capítulo 3 repasa el estado del arte de la decodificación del habla a partir de actividad neuronal y de los modelos de habla preentrenados. El capítulo 4 presenta el conjunto de datos utilizado. El capítulo 5 explica los fundamentos de los modelos y técnicas empleados. El **??** describe el sistema construido a partir de OWSM y las adaptaciones que se le hacen. El capítulo 6 detalla cómo se entrenan y evalúan los modelos y el diseño de los dos bloques de experimentos. El capítulo 7 recoge y discute los resultados. El capítulo 8 reúne las limitaciones del estudio y las líneas de trabajo futuro y, por último, el capítulo 9 resume las conclusiones. Al final se incluyen los anexos, entre ellos el de la relación del trabajo con los Objetivos de Desarrollo Sostenible.

CAPÍTULO 3
Estado del arte

Este capítulo repasa lo que se ha hecho hasta ahora en la decodificación del habla a partir de la actividad cerebral y en la reutilización de modelos de habla. Primero se comparan las técnicas de registro y se recogen las cifras de las neuroprótesis del habla más recientes (secciones 3.1 y 3.2). Después se explica cómo están construidos los decodificadores actuales y por qué (sección 3.3), y qué resultados se obtienen en el *benchmark* Brain-to-Text '25 (sección 3.4). Por último se resumen los modelos de habla preentrenados y los trabajos que los han llevado a la señal neuronal (sección 3.5), se comparan todos los sistemas vistos (sección 3.6) y se sitúa este trabajo respecto a ellos (sección 3.7). En el capítulo aparecen algunos conceptos, como las métricas de error o las partes de un modelo, que se explican más adelante, en el marco teórico (capítulo 5).

3.1 Modalidades de registro en interfaces cerebro-computador

La técnica de registro condiciona qué información se puede extraer y qué tipo de decodificador tiene sentido construir. Las técnicas utilizadas en BCI del habla van de menos a más invasivas. En general, cuanto más invasiva es la técnica, mejor es la señal:

Electroencefalografía (EEG). Es una técnica no invasiva, barata y fácil de conseguir. Registra, a través del cráneo, la actividad sumada de grandes grupos de neuronas, por lo que la señal tiene mucho ruido. Funciona bien para elegir entre pocas opciones, pero la decodificación de texto a partir de EEG está en discusión: se ha visto que algunos modelos publicados rinden casi igual utilizando solo ruido que utilizando EEG [40]. Además, suele presentar problemas de portabilidad y comodidad, y es la técnica más afectada por el paso del tiempo.

Estereoelectroencefalografía (sEEG). Utiliza electrodos de profundidad que se implantan con fines diagnósticos en clínicas, a través de pequeños orificios y sin necesidad de abrir el cráneo [41]. Tiene buena resolución temporal y permite llegar a estructuras profundas, pero la colocación de los electrodos responde a criterios clínicos y no a las necesidades del decodificador. Por ello, no suele utilizarse para este tipo de problemas.

Electrocorticografía (ECoG). Utiliza rejillas de electrodos colocadas sobre la superficie de la corteza, lo que obliga a abrir el cráneo [42]. Estas rejillas cubren una zona amplia, son estables durante mucho tiempo y han permitido construir los primeros sistemas de habla en tiempo real [17], [18].

Matrices intracorticales de microelectrodos. Son matrices de electrodos que penetran en la corteza, normalmente de tipo Utah, y registran la actividad de pequeños grupos

de neuronas cercanos a cada electrodo. Es la técnica con mayor riesgo quirúrgico, pero con ella se han conseguido prestaciones propias de una conversación [23], [24]. Es la técnica utilizada para obtener los datos de este trabajo, cuyas características se describen en el capítulo 4.

La tabla 3.1 resume las cuatro.

Tabla 3.1: Comparación de las modalidades de registro usadas en BCI del habla.

Modalidad	Cirugía	Resolución espacial	Cobertura	Duración de los registros
EEG	No	Baja	Toda la cabeza, solo superficie	Sesiones de laboratorio
sEEG	Mínima, sin abrir el cráneo [41]	Alta, en puntos dispersos	Dispersa, incluye estructuras profundas	Lo que dura el ingreso clínico
ECoG	Apertura del cráneo [42]	Media	Una zona amplia de la superficie	Larga: unas 81 semanas en [17]
Intracortical	Apertura del cráneo e implante penetrante	Muy alta: grupos pequeños de neuronas	Pequeña: $3{,}2 \times 3{,}2$ mm por matriz [24]	Larga: 20 meses en Brain-to-Text '25 [30]

3.2 Evolución de las neuroprótesis del habla

En esta sección se recogen los trabajos que marcaron cada salto en el campo de las neuroprótesis. Es importante entenderlos para poder justificar decisiones más tarde.

Uno de los primeros sistemas que produjo lenguaje de forma continua decodificaba la escritura: el participante intentaba escribir en vez de hablar. Llegó a unos 90 caracteres por minuto, con más de un 99 % de acierto usando un autocorrector [22].

Con el habla intentada, Moses *et al.* decodificaron frases en tiempo real con ECoG en una persona con anartria, con un vocabulario de 50 palabras y una tasa de error por palabra (WER) del 25,6 % [17]. En 2023, dos trabajos pasaron a vocabularios mucho más grandes. Metzger *et al.*, con ECoG de alta densidad y también síntesis de voz y un avatar, llegaron a un 25 % de WER sobre 1.024 palabras [18]. Willett *et al.*, con matrices intracorticales, llegaron a un 23,8 % sobre 125.000 palabras [23].

El sistema de Card *et al.*, del que proceden los datos de este trabajo, es el referente actual. Con 30 minutos de datos de calibración alcanzó un 99,6 % de acierto con 50 palabras el primer día, y un 90,2 % con 125.000 palabras el segundo día, tras 1,4 horas más de datos. Mantuvo un 97,5 % de acierto durante 8,4 meses, y el participante lo usó para conversar más de 248 horas [24]. Su decodificador combina una red recurrente que predice fonemas con un modelo de n-gramas y un modelo de lenguaje de gran tamaño que reordena las frases candidatas.

Desde entonces el campo se ha abierto mucho. Los datos de Willett *et al.* y de Card *et al.* se publicaron en los retos Brain-to-Text '24 y '25 [29], [30], y ahora muchos grupos sin acceso a pacientes trabajan sobre los mismos registros e intentan mejorar sus cifras. En el reto de 2025, el sistema de referencia de los organizadores tenía un 6,7 % de WER, y los primeros puestos bajaron hasta cerca del 1,5 % juntando varios decodificadores y modelos de lenguaje de gran tamaño [30]. Debemos tener en cuenta que estos resultados se obtienen en diferido, sobre datos ya grabados, y no se han probado en tiempo real con el participante, como sí se hizo con el sistema de Card.

3.3 Anatomía de un decodificador *brain-to-text*

Los sistemas anteriores siguen casi todos el mismo esquema, heredado del reconocimiento de voz clásico, y lo van mejorando: un codificador neuronal convierte la señal en probabilidades de fonemas o palabras y un modelo de lenguaje forma las frases [43]. Esta sección explica por qué se ha llegado a ese esquema y hacia dónde está cambiando.

3.3.1. El cambio entre días

Lo que más condiciona el diseño es que la señal cambia de un día a otro, así que un decodificador entrenado con los datos de un día va perdiendo precisión [44]. La respuesta del campo ha ido cambiando. Willett *et al.* añadieron delante de un modelo compartido una capa de entrada propia de cada día [22], una solución (sección 5.2.2) que se ha mantenido en los sistemas posteriores [23], [24]. Como cada día nuevo necesita datos etiquetados para su capa, el siguiente paso fue reducir esa calibración: Card *et al.* calibran el sistema con 30 minutos de datos [24], y Fan *et al.* lo recalibran con las propias frases que decodifica, sin pedir datos nuevos al usuario [37]. Los sistemas más recientes comparten parámetros entre días cercanos [45].

3.3.2. El codificador

El codificador de referencia sigue siendo una red recurrente, la de los sistemas de Willett *et al.* y Card *et al.* y la del *benchmark* [23], [24], [30]. Se mantiene porque ha costado mucho mejorarla: en el Brain-to-Text '24, otras arquitecturas más recientes no consiguieron superarla [29]. Los avances recientes llegan cuando se añade otra fuente de información: preentrenar el codificador con datos neuronales de muchas tareas, personas e incluso especies [43], partir de un modelo de voz preentrenado [45] o usar arquitecturas de reconocimiento de voz como Conformer [46].

3.3.3. La salida y el modelo de lenguaje

La unidad de salida más usada es el fonema. La corteza motora controla la articulación y no la ortografía, y con unos 40 fonemas el problema es más sencillo que con miles de palabras [23].

Pero los fonemas no son texto. Para pasar a palabras hace falta un diccionario de pronunciación, que indica qué fonemas forman cada palabra, y un modelo de lenguaje. En este paso está buena parte del rendimiento final. Los sistemas de referencia usan un modelo de n-gramas, que estima la probabilidad de cada palabra a partir de las anteriores [47], y reordenan sus resultados con modelos de lenguaje grandes.

Este esquema es caro. Por ejemplo, un modelo de 5-gramas necesita unos 300 GB de memoria y cada decodificación tarda unos 750 ms, frente a menos de 2 GB y unos 50 ms de un sistema de extremo a extremo [45]. Además, el codificador y el modelo de lenguaje se entrenan por separado y no se pueden optimizar juntos [43]. Por eso están apareciendo sistemas de extremo a extremo, que pasan de la señal al texto con un solo modelo, prediciendo directamente letras o palabras [45], [46]. Todavía quedan por detrás de los sistemas con modelo de lenguaje (sección 3.4), pero son más baratos y sencillos de usar y están aumentando rápidamente su precisión.

3.4 El *benchmark* Brain-to-Text '25

Los *benchmarks* públicos, con particiones fijas y una métrica común, han acelerado mucho el campo. El Brain-to-Text '25, el que se usa en este trabajo, se describe en el capítulo 4. Su sistema de referencia, la red recurrente con fonemas y un modelo de 5-gramas, consigue un 6,7 % de WER [45]. Las primeras posiciones se consiguen con modelos de lenguaje y, a menudo, combinando varios modelos, como ya pasó en la edición anterior [29]. BrainWhisperer, por ejemplo, quedó cuarto de 466 equipos con una versión ampliada de su sistema con modelo de 5-gramas [45].

Estas cifras se consiguen con sistemas grandes y lentos. Combinan varios decodificadores con modelos de lenguaje de gran tamaño, y solo el modelo de lenguaje de referencia de la competición necesita cientos de gigabytes de memoria [30]. Hoy sería difícil llevar algo así a un dispositivo que funcione en tiempo real y en casa del usuario. Aun así, marcan hasta dónde se puede llegar con estos datos y por eso es importante tenerlos en cuenta.

3.5 Modelos de habla preentrenados y transferencia a la señal neuronal

Por último, el reconocimiento de voz ha pasado a preentrenar modelos con enormes cantidades de audio y a adaptarlos después a cada tarea. Algunos, como wav2vec 2.0 [48], aprenden de audio sin transcribir y necesitan después un ajuste supervisado. Otros, como Whisper [36], se entrenan directamente con cientos de miles de horas de audio transcrito. Whisper es un modelo codificador-decodificador, así que, además de leer el audio, trae un decodificador que ha aprendido cómo son las frases. En *brain-to-text* eso interesa, porque el conocimiento del lenguaje es justo lo que le falta a un codificador entrenado con pocos datos. Su inconveniente para la investigación es que no se conocen sus datos ni todo su entrenamiento.

Llevar estos modelos a la actividad cerebral se ha probado de varias formas. Algunos trabajos toman del modelo de voz su arquitectura o su objetivo de entrenamiento, pero no los pesos aprendidos con audio. BrainTalker proyecta la señal de ECoG al espacio de wav2vec 2.0 para sintetizar voz con muy pocos datos [49], y Yuan y Makin entrenan wav2vec con ECoG sin etiquetar y mejoran así un decodificador supervisado, incluso pasando de un paciente a otro [50].

Otros transfieren los pesos aprendidos con audio. Fiedler *et al.* sustituyen el extractor de audio de wav2vec 2.0 por otro pensado para señal neuronal y comparan, con los datos del participante, el ajuste completo de los pesos preentrenados, el entrenamiento desde cero y el modelo preentrenado congelado. El ajuste completo da el mejor resultado, con un 18,54 % de error por carácter y un 30,97 % de WER sin modelo de lenguaje, unos 20 puntos de error por carácter mejor que desde cero [51]. BrainWhisperer lleva la idea a Whisper y a los datos del Brain-to-Text '25. Adapta la entrada y ajusta el decodificador de Whisper, entre otros cambios, y llega a un 10,2 % de WER.

3.6 Comparativa de los sistemas revisados

La tabla 3.2 reúne los sistemas que han aparecido en este capítulo. No todas las cifras se pueden comparar entre sí. Los sistemas clínicos se evaluaron en tiempo real, cada uno con su participante y su vocabulario, mientras que el resto se evalúa en diferido. Pero podemos usar estos datos para discutir los resultados de este trabajo.

Tabla 3.2: Resumen de los sistemas revisados en este capítulo.

Trabajo	Registro	Participante	Vocabulario	WER	Velocidad
Sistemas clínicos, en tiempo real					
Moses *et al.* [17]	ECoG	Anartria	50 palabras	25,6 %	15 ppm
Willett *et al.* [22]	Intracortical	Tetraplejia	Escritura	<1 %*	90 cpm
Metzger *et al.* [18]	ECoG	Anartria	1.024 palabras	25 %	78 ppm
Willett *et al.* [23]	Intracortical	ELA (T12)	125.000 palabras	23,8 %	62 ppm
Card *et al.* [24]	Intracortical	ELA (T15)	125.000 palabras	$\sim$2,5 %	32 ppm
Brain-to-Text '25 con modelo de lenguaje externo, en diferido					
Referencia de la competición [30]	Intracortical	ELA (T15)	125.000 palabras	6,7 %†	–
BrainWhisperer [45]	Intracortical	ELA (T15)	125.000 palabras	5,9 % (5,2 %)†	–
Primeros puestos [30]	Intracortical	ELA (T15)	–	$\approx$1,5 %†	–
Brain-to-Text '25 de extremo a extremo, sin modelo de lenguaje externo, en diferido					
Red recurrente de referencia [45]	Intracortical	ELA (T15)	Abierto	16,0 %†	–
BrainWhisperer [45]	Intracortical	ELA (T15)	Abierto	10,2 % (8,7 %)†	–
Khanday *et al.* [46]	Intracortical	ELA (T15)	Abierto	23,8 %‡	–

ppm: palabras por minuto; cpm: caracteres por minuto. Entre paréntesis, entrenando también con datos de otros participantes (BrainWhisperer) o combinando varios modelos (BIT). *Error por carácter, en diferido y con autocorrector. †Partición de prueba de la competición. ‡Error por carácter en la partición de validación.

3.7 Posicionamiento de este trabajo

De esta revisión salen tres huecos, que son los que aborda este trabajo. Primero, la transferencia desde modelos de habla funciona en algunos casos, pero no se sabe bien qué parte la explica. Fiedler *et al.* usan un modelo que solo tiene codificador y otro participante, y BrainWhisperer cambia muchas cosas a la vez para conseguir la mejor cifra. Falta separar, con un mismo presupuesto, qué aporta cada parte de un modelo codificador-decodificador (el *frontend*, el codificador y el decodificador), y hacerlo con un modelo abierto como OWSM.

Segundo, en el *benchmark* las frases de validación y de prueba son de días que también tienen frases de entrenamiento (sección 4.2), así que las cifras publicadas no dicen cómo se comporta un modelo en un día nuevo. Los trabajos que estudian el paso del tiempo lo hacen con sistemas recurrentes y modelo de lenguaje [37]. Khanday *et al.* señala que el cambio entre sesiones es lo que más hace variar su error. Falta ver cómo se comporta en días futuros un sistema de extremo a extremo basado en un modelo de voz, y cuántos datos necesita.

Por último, las primeras posiciones necesitan muchos recursos. Este trabajo se limita a una GPU, un solo modelo y ningún modelo de lenguaje externo, así que su referencia son los sistemas de extremo a extremo de la tabla 3.2 y no la clasificación. Aun así, parte con desventajas conocidas frente a ellos: no usa datos de otros participantes y su presupuesto de cómputo es pequeño. Son técnicas que se pueden añadir después sobre el mismo sistema, por tanto la distancia mide lo que queda por ganar.

El trabajo más cercano es el de Khanday *et al.*, que usa los mismos datos y un Conformer entrenado desde cero que predice letras directamente, sin modelo de lenguaje, y obtiene un 23,80 % de error por carácter en validación [46]. Este trabajo sigue la misma idea de extremo a extremo, pero parte de un modelo de voz preentrenado aprovechando su decodificador.

CAPÍTULO 4
Conjunto de datos

En este capítulo se explica con qué datos se trabaja. Primero se cuenta por qué se eligió el conjunto Brain-to-Text '25. Después se describe cómo se registraron sus datos, cómo están organizados y, por último, qué tipo de frases contiene.

4.1 Selección del conjunto de datos

OWSM es un modelo de reconocimiento de voz: recibe una secuencia de varios segundos y la convierte en texto. Para adaptarlo, lo ideal es un conjunto con frases completas en lugar de palabras sueltas, y con su transcripción. También hacen falta bastantes ejemplos, porque el modelo tiene unos 100 millones de parámetros. Por último, ayuda mucho que existan particiones fijas y resultados publicados con los que comparar.

Otros aspectos son más discutibles. El primero es el tipo de señal. El EEG de superficie no necesita cirugía, así que es fácil encontrar conjuntos con muchos participantes, incluso en español. Es el caso de UGR-MINDVOICE [52], grabado en la Universidad de Granada con 15 hablantes nativos que pronuncian sílabas y palabras en voz alta y también de forma imaginada. El problema es la señal EEG, que llega con bastante ruido, y los estímulos son sílabas y palabras aisladas. Además, si el modelo fallara con EEG, no se podría saber si el problema está en OWSM o en una señal que no lleva suficiente información sobre el habla.

Con electrodos implantados (sEEG) la señal es mucho más limpia, pero hay pocos participantes porque suelen ser pacientes a los que se colocan electrodos durante su ingreso para localizar el origen de algun problema. De este tipo son sEEG2Speech [42] y SingleWordProductionDutch [41], los dos en neerlandés. El primero es el único de con frases: tres pacientes leen 100 frases de cinco a siete palabras. El segundo tiene más participantes, diez, y más de mil electrodos en total, lo que lo hace interesante para comparar entre personas, pero solo contiene palabras sueltas. En los dos casos hay entre 5 y 20 minutos de grabación por paciente, poco para los propósitos propuestos. Además, están pensados para reconstruir el audio de la voz a partir de la señal, mientras que aquí lo que se busca es texto.

Brain-to-Text '25 [24], [30] tiene la limitación contraria: un solo participante, que además habla inglés. A cambio, la señal se registra con 256 microelectrodos implantados en la corteza, que dan la señal de más calidad de las cuatro opciones, y hay cerca de 11.000 frases transcritas, grabadas en 45 sesiones a lo largo de 20 meses. Tiene particiones oficiales y una competición pública con resultados publicados, y los datos vienen ya preparados y con código para cargarlos. Es el único que cumple lo que se buscaba y por eso es el que se usa en este trabajo. Que abarque tantos meses es otra ventaja, porque permite estudiar cómo se comporta el modelo con el paso del tiempo, que es uno de los objetivos del trabajo

(OE4 en la sección 2.2). Tener un solo participante limita las conclusiones a esa persona, pero encaja con el planteamiento intrapersonal del trabajo (sección 2.3).

La tabla 4.1 resume las cuatro opciones.

Tabla 4.1: Comparación de los conjuntos de datos considerados.

	UGR-MINDVOICE [52]	**sEEG2Speech** [42]	**SingleWord-ProductionDutch** [41]	**Brain-to-Text '25** [24], [30]
Señal	EEG de superficie	sEEG (111–127 contactos por paciente)	sEEG (54–127 electrodos por paciente, 1.103 en total)	Microelectrodos intracorticales (4 matrices, 256 electrodos)
Participantes	15 hablantes nativos	3 pacientes con epilepsia	10 pacientes con epilepsia	1 paciente con ELA
Idioma	Español	Neerlandés	Neerlandés	Inglés
Material	Sílabas y palabras aisladas, en voz alta e imaginadas	100 frases de 5 a 7 palabras; 10–20 min por paciente	100 palabras aisladas; 5 min por paciente	10.948 frases de habla intentada en 45 sesiones (20 meses)
Orientación	Neurociencia e interfaces de habla silenciosa	Síntesis de voz	Producción del habla y síntesis de voz	Decodificación a texto
Acceso	OSF	OSF	OSF (iBIDS/NWB)	Dryad (HDF5 por sesión)

4.2 El conjunto Brain-to-Text '25

4.2.1. Origen y participante

Los datos proceden del ensayo clínico BrainGate2 descrito por Card *et al.* [24] y se publicaron más tarde como *benchmark* abierto y como competición [30]. El participante, identificado como T15, es un hombre de 45 años con ELA, tetraparesia y una disartria grave. Los electrodos se le implantaron cinco años después del inicio de la enfermedad, y la recogida de datos empezó en agosto de 2023, 25 días después de la operación [24].

La tarea es de copia. En la pantalla aparece una frase y, tras una señal de inicio, T15 intenta pronunciarla a su ritmo mientras se registra su actividad neuronal. No hay audio con el que alinear la señal, así que no se sabe en qué momento dice cada palabra.

4.2.2. Adquisición de la señal

La señal se registra con cuatro matrices de microelectrodos de tipo Utah implantadas en el giro precentral izquierdo. Cada matriz mide 3,2 $\times$ 3,2 mm, tiene 64 electrodos en una rejilla de 8 $\times$ 8 y se inserta 1,5 mm en la corteza, así que en total hay 256 electrodos. Se colocaron en cuatro zonas: el área 55b, relacionada con el lenguaje, y tres áreas que controlan los articuladores del habla, que son las partes dorsal y ventral del área premotora 6v y la corteza motora primaria (área 4) [24].

De cada electrodo se obtienen dos características, de modo que cada instante se describe con un vector de 512 valores:

Cruces de umbral (*threshold crossings*). Número de veces que la señal filtrada cruza un umbral fijado en $-4{,}5$ veces el valor eficaz del ruido del canal. Da una idea de la actividad de las neuronas cercanas al electrodo sin tener que separar las espigas de cada una.

Potencia en la banda de espigas (*spike band power*). Energía de la señal en la banda de frecuencias de las espigas. Complementa a la anterior y depende menos del umbral elegido.

Las dos se agregan en ventanas de 20 ms, lo que da una secuencia a 50 Hz, y se normalizan con una puntuación z dentro de cada bloque de grabación [39]. En este trabajo se usan tal y como se publican.

La figura 4.1 resume el sistema: las matrices de microelectrodos registran la actividad de la corteza mientras T15 intenta hablar, y esa señal se procesa para obtener el texto de la frase.

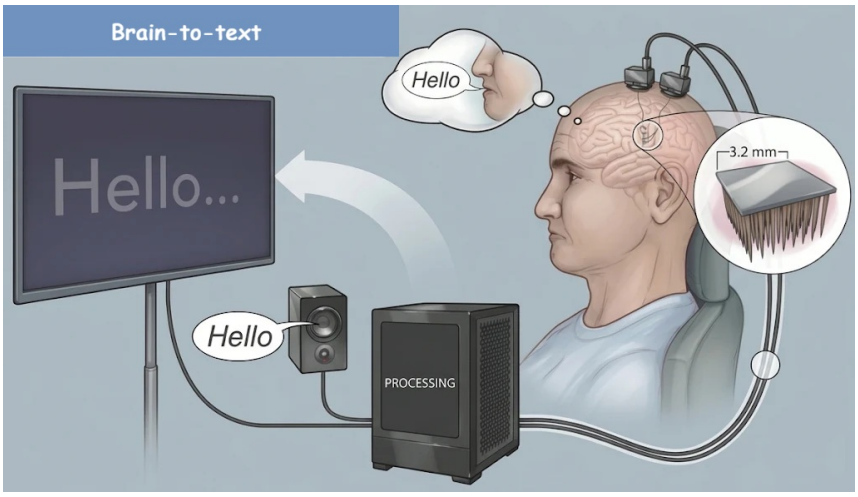


Figura 4.1: Esquema de una interfaz cerebro-texto. Las matrices de microelectrodos de tipo Utah ($3{,}2 \times 3{,}2$ mm) se implantan en la corteza motora del habla; la señal que registran mientras el participante intenta hablar se procesa para mostrar la frase como texto en una pantalla.

4.2.3. Organización de los datos

Los datos se reparten en ficheros HDF5 por sesión. Cada sesión se identifica por su fecha (`t15.AAAA.MM.DD`) y tiene hasta tres ficheros, uno por partición: entrenamiento, validación y prueba. No todas las sesiones tienen las tres. Dentro de cada fichero, cada ensayo (una frase) se guarda por separado con estos elementos [30]:

`input_features`**.** Matriz de $T \times 512$ con las características neuronales, donde T es el número de ventanas del ensayo.

`seq_class_ids`**.** Secuencia de fonemas de la frase.

`transcription`**.** Transcripción de la frase.

Atributos. Número de ventanas, longitud de la secuencia de fonemas, texto de la frase e identificadores de sesión, bloque y ensayo.

Las 45 sesiones van del 11 de agosto de 2023 al 13 de abril de 2025, y en ese tiempo la señal no se mantiene igual. Lo que capta cada electrodo cambia de un día a otro, e

incluso dentro del mismo día por movimiento de los electodos. Es lo que se conoce como no estacionariedad de la señal. La normalización por bloque corrige una parte de estos cambios, pero no todos. A esto se suma que en unos bloques T15 intenta hablar en voz alta y en otros sin voz, y los organizadores no indican, a propósito, cuál es cuál [30]. Con todo esto, no es lo mismo evaluar el modelo en un día del que ya ha visto frases que en un día nuevo, y es lo que se estudia en el segundo bloque de experimentos (capítulo 6).

4.2.4. Particiones y composición

El conjunto tiene 8.072 ensayos de entrenamiento, 1.426 de validación y 1.450 de prueba [39]. Las frases se graban en bloques de entre 5 y 50, y los bloques no se mezclan entre entrenamiento y evaluación: 198 bloques de los cual 45 sesiones, forman el entrenamiento, y los 67 restantes, de 41 sesiones, forman la validación y la prueba.

Son aproximadamente 8.000 frases de entrenamiento, todas de la misma persona. A pesar de ser un dataset grande para BCI, los modelos se suelen entrenar con muchas más horas de audio. Esto condiciona qué partes del modelo se reutilizan y cómo se entrena (capítulo 6).

Las frases proceden de seis corpus distintos (tabla 4.2). La mayoría son de Switchboard [53], un corpus de conversaciones telefónicas en inglés, seguidas de un corpus de palabras frecuentes preparado por los autores y de frases con un vocabulario cerrado de 50 palabras [30]. Tres corpus aparecen solo en validación y prueba: OpenWebText, con texto de páginas web; las frases Harvard, pensadas para cubrir los sonidos del inglés de forma equilibrada [54], y secuencias de palabras al azar. Suman 349 frases de un tipo que el modelo no ve al entrenar. La validación tiene, por tanto, frases de otro estilo y palabras que no salen en el entrenamiento, y las secuencias al azar no tienen ninguna estructura gramatical que el modelo pueda aprovechar.

Tabla 4.2: Composición de Brain-to-Text ’25 por corpus de origen, según [39]. Estos recuentos suman algunas frases más (11.162) que las tres particiones juntas (10.948).

Corpus	Bloques	Frases	Partición
Switchboard	182	7.672	Entrenamiento y validación/prueba
Palabras frecuentes	48	2.050	Entrenamiento y validación/prueba
Vocabulario de 50 palabras	25	1.091	Solo entrenamiento
OpenWebText	5	172	Solo validación/prueba
Frases Harvard	3	98	Solo validación/prueba
Palabras al azar	2	79	Solo validación/prueba

Las transcripciones de la partición de prueba no son públicas. La clasificación de la competición se calcula con ellas y, como este trabajo no participó, no es posible evaluar con esa partición. Por eso aquí la validación oficial hace de conjunto de prueba. Hay indicios de que esto apenas cambia las cifras: LightBeam, por ejemplo, obtiene un WER del 6,36 % en validación y del 5,77 % y el 6,47 % en las partes pública y privada de la prueba [39]. Cómo se usa cada partición en los experimentos se explica en el capítulo 6. No obstante, se mencionará cuando los resultado se comparen al conjunto de validación o al de prueba.

4.3 Análisis exploratorio

Este análisis es breve y se centra en las frases. Las características neuronales no se estudian aquí, porque vienen ya calculadas y normalizadas con el procedimiento habitual en la literatura [23], [24] y este trabajo se centra en el modelo.

Casi siete de cada diez frases son de Switchboard (tabla 4.2), así que el estilo que más se repite es el de una conversación espontánea. Las frases tienen de media 6,2 palabras, y la mayoría tiene entre 4 y 9 (figura 4.2). La distribución es muy parecida en entrenamiento y validación, aunque en validación hay algo más de frases largas.

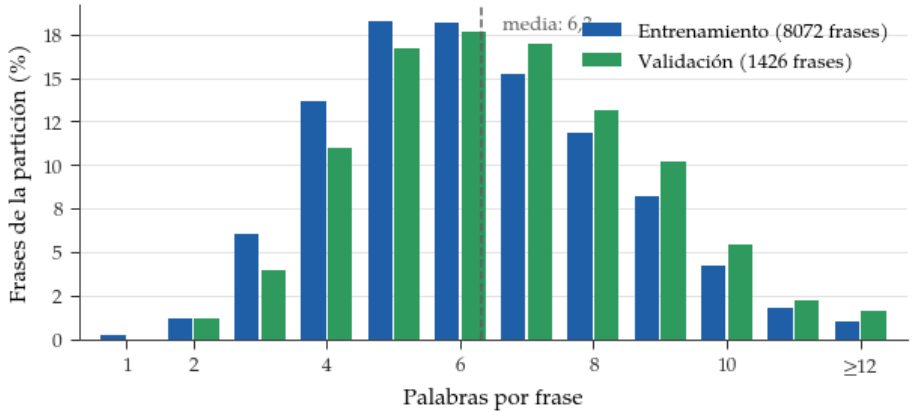


Figura 4.2: Número de palabras por frase en las particiones de entrenamiento y validación. La línea discontinua marca la media.

Las frases casi no se repiten. La mayoría aparece una sola vez en todo el conjunto (figura 4.3a), y solo el 11,4 % de las frases de validación está también, idéntica, en el entrenamiento. La figura 4.3b muestra además cuánto se parece cada frase de validación a la más parecida del entrenamiento: lo habitual es que compartan alrededor de la mitad. El modelo, por tanto, se evalúa casi siempre con frases nuevas.

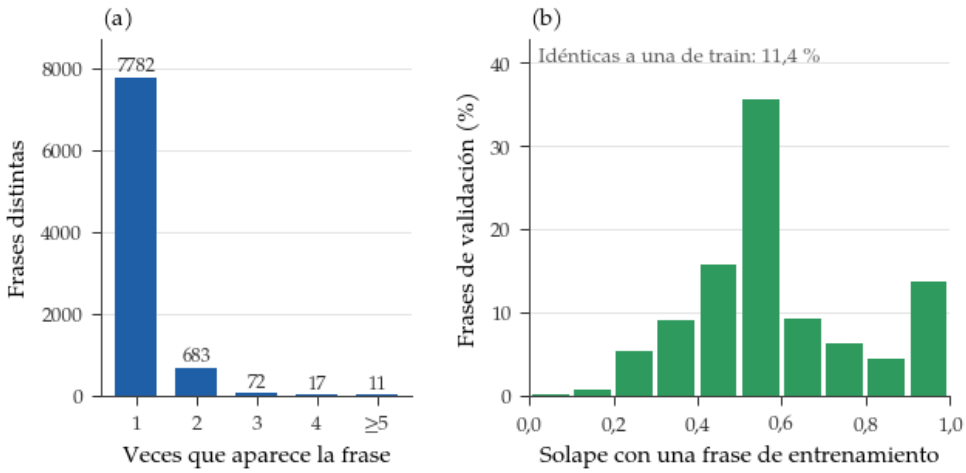


Figura 4.3: Repetición de frases y solape entre particiones. (a) Número de frases distintas según las veces que aparecen en el conjunto. (b) Solape de cada frase de validación con la frase de entrenamiento más parecida; un solape de 1 indica que la frase está tal cual en el entrenamiento.

Con las palabras ocurre lo normal en cualquier texto: unas pocas aparecen muchísimo y la mayoría muy poco. Las más frecuentes son pronombres, artículos y preposiciones, como *I*, *the*, *to* o *you* (figura 4.4a), que son las más usadas en una conversación normal. El vocabulario de entrenamiento tiene 3.745 palabras, pero las 100 más frecuentes ya cubren el 61 % del texto (figura 4.4b). Aun así, es importante ver que un [X] % de las palabras de validación no aparece nunca en el entrenamiento. El modelo solo puede escribir estas palabras nuevas relacionándolas con el contexto y usando su conocimiento de los audios.

Por último, las particiones comparten días. Todas las sesiones tienen frases de entrenamiento, así que cada frase de validación tiene otras grabadas el mismo día en el

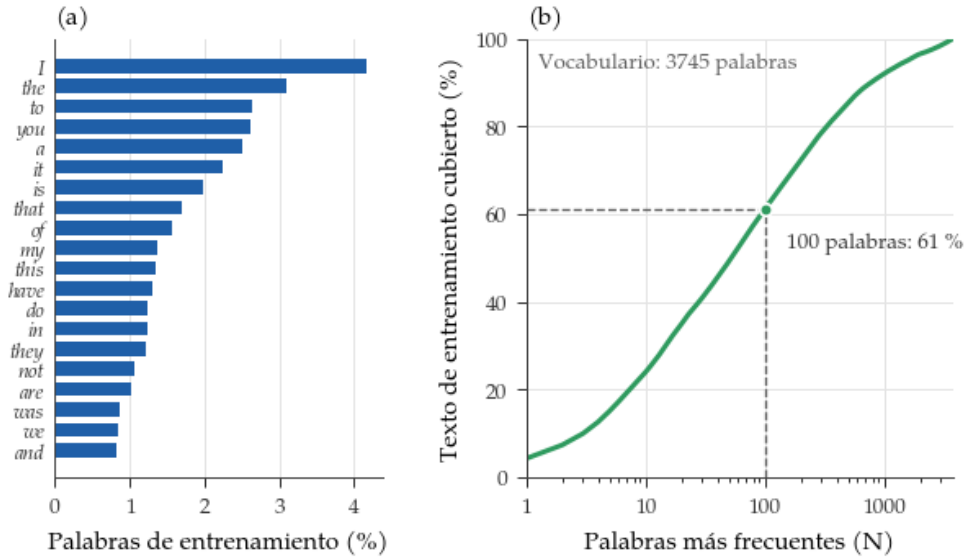


Figura 4.4: Frecuencia de las palabras en el entrenamiento. (a) Las 20 palabras más frecuentes, como porcentaje del total de palabras. (b) Porcentaje del texto de entrenamiento que cubren las N palabras más frecuentes (eje horizontal en escala logarítmica).

entrenamiento, aunque en bloques distintos. Es una situación optimista si se compara con el uso real, en el que el sistema tiene que funcionar en días que todavía no ha visto. Por eso, además de la evaluación con todas las sesiones, se hace una evaluación en orden cronológico (capítulo 6). La figura 4.5 muestra cuántas frases hay en cada sesión a lo largo de los 20 meses; las cuatro sesiones marcadas no tienen frases de validación.

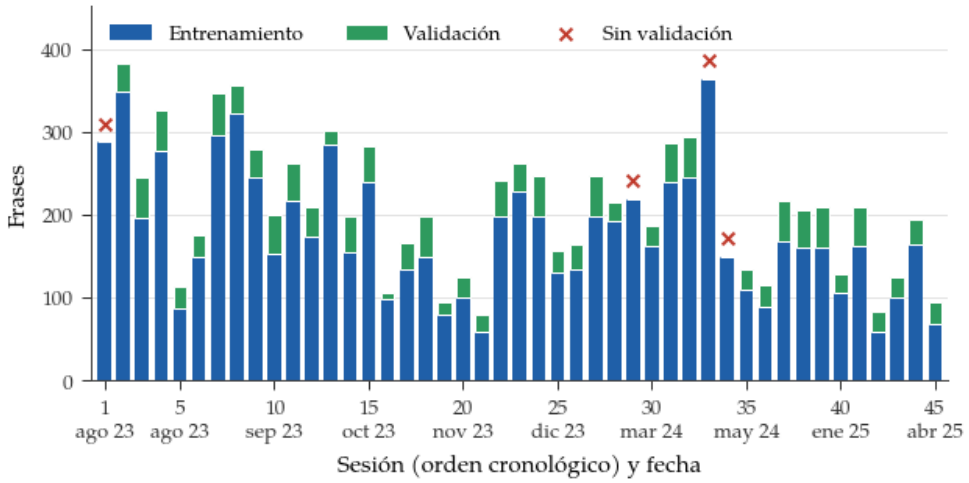


Figura 4.5: Número de frases de entrenamiento y validación en cada una de las 45 sesiones, en orden cronológico. Las cruces marcan las sesiones sin frases de validación.

CAPÍTULO 5
Marco teórico

En este capítulo se explican los modelos y técnicas que forman el sistema: qué hace cada uno y para qué sirve. Cómo se han combinado, con qué valores y qué se ha probado se cuenta en el capítulo 6.

5.1 Planteamiento del problema

Sea $\mathbf{X} = (\mathbf{x}_1, \ldots, \mathbf{x}_T)$ una secuencia de T vectores de características neuronales, con $\mathbf{x}_t \in \mathbb{R}^{512}$, obtenida durante un intento de habla, y sea $\mathbf{y} = (y_1, \ldots, y_L)$ la secuencia de unidades de texto de la frase que el participante intentaba pronunciar, con $y_l \in \mathcal{V}$. En este trabajo, $\mathcal{V}$ es el vocabulario de subpalabras de OWSM (sección 5.4.2).

El problema consiste en aprender una función $f_\theta : \mathbb{R}^{T\times 512} \to \mathcal{V}^*$ que estime $\mathbf{y}$ a partir de $\mathbf{X}$. Presenta tres características que condicionan las decisiones del trabajo:

(a) **Es una tarea de transducción de secuencias de longitud variable**, con $T \gg L$: a 50 Hz, una frase de pocos segundos produce cientos de ventanas temporales frente a unas decenas de unidades de salida.

(b) **No se dispone de alineamiento** entre las posiciones de $\mathbf{X}$ y las de $\mathbf{y}$. Al no existir audio de referencia, no puede saberse qué ventanas corresponden a cada unidad.

(c) **El orden se preserva**, esto es, la correspondencia entre entrada y salida es monótona: a diferencia de la traducción automática, la unidad y_l nunca precede temporalmente a y_{l-1}.

La combinación de (b) y (c) es el escenario para el que se diseñó la clasificación temporal conexionista, descrita en la sección 5.5.

5.2 Datos de entrada al modelo

5.2.1. Lotes, relleno y máscaras

Durante el entrenamiento, las frases se procesan en lotes de B frases a la vez, porque así se aprovecha mejor la GPU. Cada frase tiene su propia longitud T_b, pero un lote se guarda como un único tensor de tamaño $B \times T_{\text{máx}} \times 512$, donde $T_{\text{máx}}$ es la longitud de la frase más larga del lote. Las demás se completan con ceros al final. Este relleno (*padding*)

no es señal y el modelo no debe tenerlo en cuenta. Para ello, junto al lote se guardan las longitudes reales y, a partir de ellas, una máscara:

$$m_{b,t} = \begin{cases} 1 & \text{si } t \leq T_b \text{ (dato real)}, \\ 0 & \text{si } t > T_b \text{ (relleno)}. \end{cases} \tag{5.1}$$

La máscara se usa en varios sitios. En la atención (sección 5.4.3), las posiciones de relleno reciben una puntuación de $-\infty$ antes de la *softmax*, así que su peso es cero y ninguna posición real las tiene en cuenta. En la pérdida, las posiciones y los tokens de relleno no se cuentan. Además, cualquier capa con término de sesgo convierte los ceros del relleno en otros valores, de modo que antes de una convolución hay que volver a ponerlos a cero multiplicando por la máscara. Por último, cuando una capa reduce el número de ventanas, las longitudes y la máscara se recalculan con la misma regla que esa capa, la de la ecuación (5.6).

5.2.2. Capa de entrada por sesión

Como se vio en el capítulo 4, la señal que registra cada electrodo cambia de un día a otro. Un mismo intento de habla puede producir una actividad algo distinta en dos sesiones, y un modelo único tendría que aprender a la vez varias correspondencias entre señal y texto. La solución habitual en el campo es poner delante del modelo una capa de entrada propia de cada sesión [22], [23]. Es una transformación afín seguida de una no linealidad acotada:

$$\mathbf{z}_t = \text{softsign}\left(\mathbf{W}_s\, \mathbf{x}_t + \mathbf{b}_s\right), \qquad \text{softsign}(u) = \frac{u}{1 + |u|}, \tag{5.2}$$

donde s es la sesión a la que pertenece la frase, $\mathbf{W}_s \in \mathbb{R}^{512\times512}$ y $\mathbf{b}_s \in \mathbb{R}^{512}$. La idea es que cada día tenga su propia corrección y que, después de ella, la señal de todos los días se parezca más. La función *softsign* deja cada valor entre -1 y 1, así que una matriz mal ajustada no puede generar valores extremos. Las matrices empiezan siendo la identidad y los sesgos, cero, de modo que al principio la capa solo aplica la *softsign*.

Cada sesión añade $512^2 + 512 = 262{,}656$ parámetros, unos 11,8 millones con las 45 sesiones. Además, la capa solo existe para los días que tienen datos de entrenamiento: para un día nuevo no hay matriz aprendida y habría que recalibrar con datos de ese día.

5.3 *Frontend*

En reconocimiento de voz se llama *frontend* a la primera etapa del modelo, la que convierte la entrada en algo que el codificador pueda procesar. En OWSM, esa etapa toma la forma de onda, calcula un espectrograma en escala mel y reduce su resolución temporal con convoluciones (sección 5.4.2). Con señal neuronal no tiene sentido calcular un espectrograma, porque la entrada ya son características, así que esa parte se sustituye por un módulo nuevo. En este trabajo se llama *frontend* a ese módulo.

El *frontend* tiene dos tareas: pasar los 512 valores de cada ventana a la dimensión d que espera el codificador (384 en OWSM base) y reducir el número de ventanas. Si reduce la secuencia por un factor s, cada vector de salida resume $20s$ ms de señal; con $s = 4$, por ejemplo, el codificador recibe un vector cada 80 ms. Menos ventanas hacen el codificador más barato, porque el coste de la atención crece con el cuadrado de la longitud, pero CTC necesita al menos una ventana por cada unidad de salida (sección 5.5).

Los *frontends* se construyen combinando unas pocas capas:

Capa lineal. Multiplica cada vector por una matriz y le suma un sesgo:

$$\mathbf{y}_t = \mathbf{W}\mathbf{x}_t + \mathbf{b}, \qquad \mathbf{W} \in \mathbb{R}^{d_{\text{sal}} \times d_{\text{ent}}}. \tag{5.3}$$

Se aplica igual en todas las ventanas, mezcla todos los electrodos entre sí y sirve para cambiar de dimensión.

Apilado de ventanas. Concatena k ventanas seguidas en un solo vector, que después pasa por una capa lineal:

$$\mathbf{u}_t = [\mathbf{x}_{st};\, \mathbf{x}_{st+1};\, \ldots;\, \mathbf{x}_{st+k-1}] \in \mathbb{R}^{512k}. \tag{5.4}$$

El paso s indica cuántas ventanas se avanza de un vector al siguiente. Con $k = s$ los grupos no se solapan y la secuencia se reduce por s. Con $k > s$ se solapan y cada vector ve más contexto; a esto se le suele llamar *patching*.

Convolución temporal. Recorre la secuencia con un filtro que mira k ventanas seguidas, el núcleo:

$$\mathbf{y}_t = \mathbf{b} + \sum_{j=0}^{k-1} \mathbf{W}_j\, \mathbf{x}_{st+j}, \qquad \mathbf{W}_j \in \mathbb{R}^{d_{\text{sal}} \times d_{\text{ent}}}. \tag{5.5}$$

Se parece al apilado, pero se pueden encadenar varias capas, y cada salida acaba viendo cada vez más contexto. Con paso $s > 1$ la convolución reduce la secuencia. Si se añaden p ventanas de relleno en cada extremo, la longitud de salida es

$$T' = \left\lfloor \frac{T + 2p - k}{s} \right\rfloor + 1. \tag{5.6}$$

Función de activación. Se aplica a cada valor después de una capa lineal o convolucional, para que el modelo pueda aprender relaciones no lineales. Las dos más usadas en el sistema son ReLU, $\text{ReLU}(u) = \max(0, u)$, y GELU [55]:

$$\text{GELU}(u) = u\,\Phi(u), \tag{5.7}$$

donde Φ es la función de distribución de una normal estándar. GELU se comporta como una ReLU suave: deja pasar los valores positivos y atenúa los negativos poco a poco.

Normalización por capas (*LayerNorm*). Normaliza cada vector con su propia media y su propia varianza [56]:

$$\text{LN}(\mathbf{h}) = \boldsymbol{\gamma} \odot \frac{\mathbf{h} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \boldsymbol{\beta}, \tag{5.8}$$

donde μ y σ^2 son la media y la varianza de los d valores de $\mathbf{h}$, ϵ es una constante pequeña y $\boldsymbol{\gamma}$ y $\boldsymbol{\beta}$ se aprenden. Mantiene los valores en una escala parecida de una capa a otra, lo que estabiliza el entrenamiento.

***Dropout*.** Durante el entrenamiento pone a cero cada valor con probabilidad p y multiplica los demás por $1/(1-p)$ [57]. Así el modelo no puede depender de unas pocas neuronas concretas. En la evaluación no se apaga nada.

Conexión residual. Suma la entrada de un bloque a su salida [58]:

$$\mathbf{y} = \mathbf{x} + F(\mathbf{x}). \tag{5.9}$$

El bloque solo tiene que aprender la corrección $F(\mathbf{x})$ y el gradiente llega mejor a las primeras capas, lo que permite apilar muchas.

Codificación posicional. La atención no sabe por sí misma en qué orden están las ventanas (sección 5.4.3). Para que lo sepa, se suma a cada vector un patrón que depende de su posición t [59]:

$$\mathrm{PE}_{t,2i} = \sin\left(\frac{t}{10000^{2i/d}}\right), \qquad \mathrm{PE}_{t,2i+1} = \cos\left(\frac{t}{10000^{2i/d}}\right). \tag{5.10}$$

Cada dimensión oscila con una frecuencia distinta, así que cada posición tiene un patrón único y las posiciones cercanas se parecen.

Qué combinación de estas capas forma cada *frontend* probado se explica en el capítulo 6.

5.4 OWSM

5.4.1. Origen y versiones

Whisper [36] es un modelo de reconocimiento de voz de OpenAI que, en su versión original, se entrenó con unas 680.000 horas de audio de internet. Es multilingüe y hace varias tareas, como transcribir, traducir o identificar el idioma. Sus pesos son públicos, pero no lo son los datos ni el proceso completo de entrenamiento. OWSM (*Open Whisper-style Speech Model*) reproduce ese tipo de entrenamiento con datos públicos y con la herramienta de código abierto ESPnet [34], y publica las recetas, el código, los registros de entrenamiento y los pesos [32]. Las primeras versiones usan un codificador y un decodificador Transformer, y OWSM v3 se entrenó con unas 180.000 horas.

OWSM v3.1 [33] cambia el codificador por un E-Branchformer (sección 5.4.4) con los mismos datos, y mejora a la vez la precisión y la velocidad. Cubre 151 idiomas y se publica en tres tamaños: *base* (101 millones de parámetros), *small* (367 millones) y *medium* (1.020 millones). OWSM v4 [60] sigue la misma línea y, sobre todo, añade datos: 166.000 horas de YODAS, un conjunto de audio de internet con licencia Creative Commons que los autores limpiaron, hasta llegar a unas 320.000 horas en total. Su versión base tiene 102 millones de parámetros. En este trabajo se usan las versiones base de v3.1 y v4, las más pequeñas de cada serie.

5.4.2. Arquitectura

OWSM es un modelo codificador-decodificador con atención, al que se añade una cabeza CTC sobre la salida del codificador. Con audio, el proceso completo es el siguiente:

1. ***Frontend* acústico.** La forma de onda se convierte en un espectrograma en escala mel y se normaliza.
2. **Submuestreo convolucional.** Dos convoluciones bidimensionales, que tratan el espectrograma como una imagen de tiempo por frecuencia, reducen la secuencia (por cuatro en OWSM v3.1). Una capa lineal la lleva a la dimensión d y se suma la codificación posicional.
3. **Codificador.** Una pila de bloques E-Branchformer transforma la secuencia en otra de la misma longitud, $\mathbf{H} = (\mathbf{h}_1, \ldots, \mathbf{h}_{T'})$, en la que cada vector describe lo que pasa en ese instante teniendo en cuenta toda la frase.
4. **Decodificador.** Un Transformer genera el texto token a token. En cada paso mira los tokens que ya ha escrito y, mediante la atención cruzada, la salida del codificador.

5. **Cabeza CTC.** Una capa lineal sobre cada $\mathbf{h}_t$ da una distribución sobre los tokens. Se usa en el entrenamiento y en la decodificación (sección 5.5).

En la versión base, el codificador y el decodificador tienen 6 capas cada uno y trabajan con vectores de dimensión $d = 384$.

El texto se representa con un vocabulario de unas 50.000 subpalabras obtenido con SentencePiece [61]. Las palabras frecuentes son un solo token y las raras se parten en varios trozos, así que el modelo puede escribir palabras que no ha visto nunca juntando trozos conocidos. Además hay tokens especiales que indican el idioma (`<eng>`), la tarea (`<asr>`, transcribir) y si se quieren marcas de tiempo (`<notimestamps>`). La secuencia que procesa el decodificador empieza con `<sos>` y estos tokens especiales, sigue con el texto de la frase y termina con `<eos>`.

5.4.3. Atención

La atención es la operación que permite a cada posición de una secuencia mirar a las demás y decidir cuáles le importan. Cada vector se proyecta, con matrices aprendidas, en tres vectores: una consulta, una clave y un valor. La consulta de una posición se compara con las claves de todas mediante un producto escalar, y cuanto más se parecen, más peso recibe el valor de esa posición. En forma matricial:

$$\text{Atención}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}, \tag{5.11}$$

donde las filas de $\mathbf{Q}$, $\mathbf{K}$ y $\mathbf{V}$ son las consultas, las claves y los valores, d_k es su dimensión y $\mathbf{M}$ es la máscara, con 0 en las posiciones permitidas y $-\infty$ en las que no se pueden mirar (sección 5.2.1). La *softmax* convierte cada fila de puntuaciones en pesos positivos que suman uno, y la división por $\sqrt{d_k}$ evita que los productos crezcan con la dimensión y saturen la *softmax*. Cada salida es, por tanto, una media ponderada de los valores.

En la autoatención, las consultas, las claves y los valores salen de la misma secuencia. En la atención cruzada, que usa el decodificador, las consultas salen del texto y las claves y los valores, de la salida del codificador. La atención multicabeza repite la operación h veces en paralelo, con proyecciones distintas de dimensión d/h, y concatena los resultados, de modo que cada cabeza puede fijarse en un tipo de relación distinto [59].

Su ventaja es que cualquier posición puede relacionarse con cualquier otra en una sola capa, por lejos que esté. Su inconveniente es el coste: compara todas las posiciones con todas, así que crece con el cuadrado de la longitud de la secuencia.

5.4.4. Codificador E-Branchformer

En el habla hay patrones a dos escalas. Los locales duran unas pocas decenas de milisegundos, como el paso de un sonido al siguiente, y los globales abarcan la frase entera. La atención capta bien lo global, pero lo local tiene que aprenderlo de los datos. Las convoluciones, al revés, captan bien lo local, pero solo ven lo que cabe en su núcleo. Por eso los codificadores de voz actuales combinan las dos: Conformer [62] las pone una detrás de otra dentro de cada bloque, Branchformer [63] las pone en dos ramas paralelas y E-Branchformer [64], el codificador de OWSM v3.1, mejora la forma de unir esas dos ramas.

Cada bloque E-Branchformer recibe una secuencia $\mathbf{E} \in \mathbb{R}^{T' \times d}$ y calcula:

$$\mathbf{E}_1 = \mathbf{E} + \tfrac{1}{2}\,\mathrm{FFN}(\mathrm{LN}(\mathbf{E})), \tag{5.12}$$

$$\mathbf{E}_G = \mathrm{MHSA}(\mathrm{LN}(\mathbf{E}_1)), \qquad \mathbf{E}_L = \mathrm{cgMLP}(\mathrm{LN}(\mathbf{E}_1)), \tag{5.13}$$

$$\mathbf{E}_2 = \mathbf{E}_1 + \mathrm{Fusión}(\mathbf{E}_G, \mathbf{E}_L), \tag{5.14}$$

$$\mathbf{E}_3 = \mathbf{E}_2 + \tfrac{1}{2}\,\mathrm{FFN}(\mathrm{LN}(\mathbf{E}_2)), \tag{5.15}$$

$$\mathbf{E}_{\mathrm{sal}} = \mathrm{LN}(\mathbf{E}_3), \tag{5.16}$$

donde LN es la normalización por capas y MHSA, la autoatención multicabeza. Las piezas son estas:

Redes *feed-forward* (FFN). Dos capas lineales con una activación σ entre ellas, aplicadas a cada vector por separado: $\mathrm{FFN}(\mathbf{h}) = \mathbf{W}_2\,\sigma(\mathbf{W}_1\mathbf{h} + \mathbf{b}_1) + \mathbf{b}_2$. La capa intermedia es más ancha que d. Hay una al principio y otra al final del bloque, cada una con peso un medio, una disposición que se conoce como *macaron*.

Rama global. La autoatención multicabeza, que relaciona cada instante con toda la frase.

Rama local (cgMLP). Un perceptrón con una puerta convolucional. Una capa lineal ensancha cada vector y aplica GELU. El resultado se parte en dos mitades, $\mathbf{Z}_1$ y $\mathbf{Z}_2$; la segunda se normaliza, pasa por una convolución en profundidad y se multiplica elemento a elemento por la primera, $\tilde{\mathbf{Z}} = \mathbf{Z}_1 \odot \mathrm{DWConv}(\mathrm{LN}(\mathbf{Z}_2))$. Por último, otra capa lineal vuelve a la dimensión d. La convolución en profundidad filtra cada canal por separado a lo largo del tiempo, así que es barata y solo mira a los vecinos de cada instante.

Fusión. Concatena las salidas de las dos ramas, $\mathbf{C} = [\mathbf{E}_G;\, \mathbf{E}_L]$, les suma una convolución en profundidad y vuelve a la dimensión d con una capa lineal: $\mathrm{Fusión}(\mathbf{E}_G, \mathbf{E}_L) = \mathbf{W}_f\,(\mathbf{C} + \mathrm{DWConv}(\mathbf{C}))$. Esa convolución es lo que añade E-Branchformer respecto a Branchformer: hace que la combinación de las dos ramas dependa también de los instantes vecinos.

Los bloques se apilan uno detrás de otro. Cada bloque mantiene la longitud y la dimensión, y la salida del último es la secuencia $\mathbf{H}$ que usan el decodificador y la cabeza CTC.

5.4.5. Decodificador Transformer

El decodificador genera la frase token a token. Para escribir el token y_l recibe los anteriores, $y_1, \ldots, y_{l-1}$, que se convierten en vectores con una tabla de *embeddings* a la que se suma la codificación posicional. Cada una de sus capas tiene tres partes, todas con conexión residual y normalización:

1. **Autoatención con máscara causal.** Cada token mira solo a los anteriores y a sí mismo. La máscara causal pone $-\infty$ en las posiciones futuras, de modo que el modelo no puede ver lo que todavía no ha escrito.
2. **Atención cruzada.** Las consultas salen del texto y las claves y los valores, de la salida del codificador $\mathbf{H}$. Es el único punto en el que el decodificador mira la señal.
3. **Red *feed-forward***, como las del codificador.

Al final, una capa lineal y una *softmax* sobre el último vector, $\mathbf{g}_l$, dan la probabilidad de cada token del vocabulario:

$$p(y_l \mid y_{<l}, \mathbf{X}) = \operatorname{softmax}\left(\mathbf{W}_o\, \mathbf{g}_l + \mathbf{b}_o\right)_{y_l}. \tag{5.17}$$

En el entrenamiento, los tokens anteriores son siempre los de la frase real, aunque el modelo se hubiera equivocado en ellos (*teacher forcing*) [65], y se minimiza la entropía cruzada:

$$\mathcal{L}_{\text{att}} = -\sum_{l=1}^{L} \log p(y_l \mid y_{<l}, \mathbf{X}). \tag{5.18}$$

En la inferencia no hay frase real: el modelo usa sus propias predicciones y sigue hasta que genera `<eos>` (sección 5.5.3).

Como cada token depende de los anteriores, el decodificador aprende qué secuencias de palabras son probables, es decir, funciona como un modelo de lenguaje interno. El de OWSM lo ha aprendido con las transcripciones de cientos de miles de horas de habla, y por eso es la parte del modelo que aporta el conocimiento del lenguaje.

5.5 Clasificación temporal conexionista (CTC)

5.5.1. Formulación y cabeza CTC

CTC [66] permite entrenar con pares $(\mathbf{X}, \mathbf{y})$ sin saber qué ventanas corresponden a cada token. Para ello añade al vocabulario un símbolo en blanco, $\varnothing$, que significa «aquí no se emite nada», y trabaja con alineamientos: secuencias $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_{T'})$ que asignan un símbolo de $\mathcal{V}' = \mathcal{V} \cup \{\varnothing\}$ a cada ventana de salida del codificador.

La cabeza CTC es la capa que da la probabilidad de cada símbolo en cada ventana. Es una capa lineal seguida de una *softmax* sobre $\mathcal{V}'$:

$$p(\pi_t \mid \mathbf{X}) = \operatorname{softmax}\left(\mathbf{W}_c\, \mathbf{h}_t + \mathbf{b}_c\right)_{\pi_t}, \qquad t = 1, \ldots, T'. \tag{5.19}$$

Un alineamiento se convierte en texto con la función de colapso $\mathcal{B}$, que primero une las repeticiones seguidas y después quita los blancos. Por ejemplo, $(\mathrm{h}, \mathrm{h}, \varnothing, \mathrm{o}, \mathrm{o})$ y $(\varnothing, \mathrm{h}, \varnothing, \varnothing, \mathrm{o})$ dan los dos `ho`. El blanco hace falta para poder escribir dos símbolos iguales seguidos: `oo` necesita un blanco entre las dos `o`. Esto también fija un mínimo: la salida del codificador tiene que tener al menos tantas ventanas como tokens tiene la frase, más una por cada par de tokens iguales seguidos.

Si se supone que, dada la entrada, cada ventana es independiente de las demás, la probabilidad de un alineamiento es el producto de las probabilidades de cada ventana, y la de la frase es la suma sobre todos los alineamientos que colapsan a ella:

$$p(\boldsymbol{\pi} \mid \mathbf{X}) = \prod_{t=1}^{T'} p(\pi_t \mid \mathbf{X}), \tag{5.20}$$

$$p(\mathbf{y} \mid \mathbf{X}) = \sum_{\boldsymbol{\pi} \in \mathcal{B}^{-1}(\mathbf{y})} p(\boldsymbol{\pi} \mid \mathbf{X}). \tag{5.21}$$

La pérdida es $\mathcal{L}_{\text{CTC}} = -\log p(\mathbf{y} \mid \mathbf{X})$. El número de alineamientos crece de forma exponencial con T', pero la suma se calcula de forma exacta, en un tiempo proporcional a $T' \cdot L$, con un algoritmo de programación dinámica hacia delante y hacia atrás.

La hipótesis de independencia es lo que hace posible el cálculo, pero también su punto débil: la predicción de cada ventana no tiene en cuenta lo que se emite en las demás, así que CTC no aprende por sí mismo qué secuencias de palabras son probables.

5.5.2. Pérdida conjunta e InterCTC

OWSM se entrena con una pérdida que combina CTC y el decodificador [67], [68]:

$$\mathcal{L} = \lambda\, \mathcal{L}_{\text{CTC}} + (1-\lambda)\, \mathcal{L}_{\text{att}}, \qquad \lambda \in [0,1]. \tag{5.22}$$

Las dos partes se ayudan. CTC obliga a que la correspondencia entre señal y texto sea monótona, y eso guía al decodificador, que por sí solo es demasiado flexible y tarda más en aprender a qué parte de la señal debe mirar. El decodificador, a su vez, aporta el contexto entre tokens que le falta a CTC.

InterCTC [69] añade otra pérdida CTC, con la misma cabeza, sobre la salida de una capa intermedia k del codificador, además de la de la última capa N:

$$\mathcal{L}_{\text{CTC}} = (1-w)\, \mathcal{L}_{\text{CTC}}^{(N)} + w\, \mathcal{L}_{\text{CTC}}^{(k)}, \qquad k < N. \tag{5.23}$$

Así las primeras capas reciben un gradiente más directo y se las obliga a que su salida ya se pueda convertir en texto, lo que funciona como regularización. Solo se usa en el entrenamiento.

5.5.3. Decodificación

Una vez entrenado, el modelo puede convertir la señal en texto de dos formas. La más sencilla usa solo la cabeza CTC: toma en cada ventana el símbolo más probable y aplica el colapso,

$$\hat{\mathbf{y}}_{\text{voraz}} = \mathcal{B}\left(\arg\max_{\pi_1} p(\pi_1 \mid \mathbf{X}), \ldots, \arg\max_{\pi_{T'}} p(\pi_{T'} \mid \mathbf{X})\right). \tag{5.24}$$

Es muy rápida, porque no necesita el decodificador, y muestra lo que ha aprendido el codificador por sí solo. Es una aproximación, ya que la frase más probable no tiene por qué corresponder al alineamiento más probable.

La segunda es la búsqueda en haz [68]. El texto se genera token a token con el decodificador, pero en cada paso se guardan las K hipótesis parciales con mejor puntuación en lugar de solo la mejor. La puntuación de cada hipótesis $\hat{y}_{1:l}$ combina el decodificador y CTC:

$$s(\hat{y}_{1:l}) = \lambda \log p_{\text{CTC}}(\hat{y}_{1:l} \mid \mathbf{X}) + (1-\lambda) \log p_{\text{att}}(\hat{y}_{1:l} \mid \mathbf{X}), \tag{5.25}$$

donde p_{att} es el producto de las probabilidades de la ecuación (5.17) y p_{CTC} es la probabilidad, según CTC, de que la frase empiece por $\hat{y}_{1:l}$. CTC penaliza las hipótesis que no encajan con la señal, por ejemplo las que repiten palabras o se alargan sin motivo, y el decodificador favorece las que tienen sentido como frase. La búsqueda acaba cuando las mejores hipótesis terminan en `<eos>`. En este trabajo no se añade ningún modelo de lenguaje externo: el conocimiento del lenguaje sale del decodificador.

5.6 Adaptación de un modelo preentrenado

Un modelo preentrenado es un modelo que ya se ha entrenado con muchos datos para una tarea, como OWSM con cientos de miles de horas de audio. Reutilizarlo en una tarea nueva se llama aprendizaje por transferencia: en lugar de empezar de cero, se parte de unos pesos que ya han aprendido algo útil y se ajustan con los datos nuevos (*fine-tuning*) [70]. Suele funcionar mejor cuanto más se parecen las dos tareas.

Entrenar un componente desde cero significa partir de pesos aleatorios. No se inicializan a cero: si todos los pesos de una capa empezaran iguales, todas sus neuronas

calcularían lo mismo y recibirían el mismo gradiente, así que nunca llegarían a diferenciarse. Por eso se sortean valores pequeños, con una escala que depende del tamaño de la capa para que las señales no crezcan ni se apaguen al atravesar la red [71]. Hay casos concretos en los que sí tiene sentido empezar con un valor fijo, como la identidad de la capa por sesión (sección 5.2.2) o la matriz **B** de LoRA.

Congelar un componente significa que sus pesos no se actualizan durante el entrenamiento. Sigue funcionando igual en la pasada hacia delante y el gradiente lo sigue atravesando hacia las capas anteriores, de modo que un decodificador congelado puede seguir indicando al codificador qué salida necesita. Descongelar es lo contrario, y se puede hacer con todo el componente o solo con algunas capas, normalmente las más cercanas a la salida. También se puede dar a cada parte del modelo una tasa de aprendizaje distinta (sección 5.7), por ejemplo más baja para lo preentrenado que para lo nuevo.

5.6.1. LoRA

LoRA (*Low-Rank Adaptation*) [38] ajusta un modelo preentrenado cambiando muy pocos parámetros. Parte de la idea de que el cambio que necesitan los pesos para la tarea nueva tiene rango bajo. Para una matriz preentrenada $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$, en lugar de aprender un cambio completo $\Delta\mathbf{W}$, se aprende como el producto de dos matrices estrechas:

$$\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r}\,\mathbf{B}\mathbf{A}, \qquad \mathbf{B} \in \mathbb{R}^{d\times r},\ \mathbf{A} \in \mathbb{R}^{r\times k},\ r \ll \min(d,k). \tag{5.26}$$

$\mathbf{W}_0$ se queda congelada y solo se entrenan **A** y **B**, así que los parámetros entrenables pasan de $d \cdot k$ a $r(d+k)$. **A** empieza con valores aleatorios y **B** a cero, de modo que al principio $\Delta\mathbf{W} = \mathbf{0}$ y el modelo se comporta exactamente como el preentrenado. El rango r controla cuánto puede cambiar la matriz y el factor α/r fija la escala del cambio. Suele aplicarse a las proyecciones de la atención, como las de las consultas y los valores. Como el cambio queda limitado a un subespacio pequeño, LoRA también actúa como regularización: el modelo no puede alejarse mucho de lo que ya sabía.

5.7 Entrenamiento

Esta sección resume los conceptos de entrenamiento que aparecen en el resto de la memoria. Los valores de cada experimento están en el capítulo 6.

Parámetros e hiperparámetros. Los parámetros son los pesos que aprende el modelo. Los hiperparámetros se fijan antes de entrenar, como la tasa de aprendizaje, el tamaño del lote o el número de épocas. Los parámetros entrenables son los que se actualizan, y pueden ser muchos menos que el total si hay partes congeladas.

Conjuntos de datos. El de entrenamiento sirve para ajustar los pesos. El de validación sirve para tomar decisiones durante el entrenamiento, como elegir la mejor época, y para detectar el sobreajuste. El de prueba solo se usa al final, para medir el resultado con datos que no han influido en ninguna decisión. A veces se aparta una parte del entrenamiento, una reserva, para que haga de validación.

Lote, paso y época. Un lote es el grupo de frases que se procesa a la vez. En cada paso se calcula la pérdida de un lote y se actualizan los pesos una vez. Una época es una pasada completa por el conjunto de entrenamiento, aunque algunas herramientas, como ESPnet, permiten fijarla como un número de pasos.

Descenso por gradiente. Entrenar es buscar los parámetros $\boldsymbol{\theta}$ que minimizan la pérdida. En cada paso se calcula el gradiente $\mathbf{g} = \nabla_{\boldsymbol{\theta}} \mathcal{L}$ mediante retropropagación y los parámetros se mueven en la dirección contraria, con un tamaño de paso dado por la tasa de aprendizaje η: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\, \mathbf{g}$.

AdamW. En la práctica se usa una versión mejorada de esa regla. Adam [72] lleva una media móvil del gradiente y otra de su cuadrado, y adapta el paso a cada parámetro. AdamW [73] le añade el decaimiento de pesos por separado. En el paso n:

$$\begin{aligned} \mathbf{m}_n &= \beta_1 \mathbf{m}_{n-1} + (1-\beta_1)\, \mathbf{g}_n, & \mathbf{v}_n &= \beta_2 \mathbf{v}_{n-1} + (1-\beta_2)\, \mathbf{g}_n^2, \\ \hat{\mathbf{m}}_n &= \frac{\mathbf{m}_n}{1-\beta_1^n}, & \hat{\mathbf{v}}_n &= \frac{\mathbf{v}_n}{1-\beta_2^n}, \\ \boldsymbol{\theta}_n &= \boldsymbol{\theta}_{n-1} - \eta_n \left(\frac{\hat{\mathbf{m}}_n}{\sqrt{\hat{\mathbf{v}}_n} + \epsilon} + \lambda_{\mathrm{wd}}\, \boldsymbol{\theta}_{n-1} \right), \end{aligned} \tag{5.27}$$

donde β_1 y β_2 controlan cuánto pesan los pasos anteriores en las medias, ϵ evita dividir por cero y λ_{wd} es el decaimiento de pesos, que empuja los pesos hacia cero y funciona como regularización.

Tasa de aprendizaje y calentamiento. Al principio del entrenamiento, los pasos grandes pueden desestabilizar el modelo, sobre todo en los Transformers. Por eso se empieza con una tasa pequeña que sube poco a poco, el calentamiento (*warmup*), y luego baja. La regla que usa ESPnet viene del Transformer original [59]:

$$\eta_n = \eta_0 \cdot w^{0{,}5} \cdot \min\left(n^{-0{,}5},\ n\, w^{-1{,}5} \right), \tag{5.28}$$

donde η_0 es la tasa máxima y w el número de pasos de calentamiento. La tasa sube en línea recta hasta η_0 en el paso w y después baja de forma proporcional a $1/\sqrt{n}$.

Recorte del gradiente. Si la norma del gradiente supera un umbral c, se reescala para que valga c: $\mathbf{g} \leftarrow c\, \mathbf{g} / \|\mathbf{g}\|$. Así un lote raro no puede dar un paso enorme y desestabilizar el entrenamiento [74].

Sobreajuste. Ocurre cuando el modelo aprende de memoria el entrenamiento y deja de generalizar: la pérdida de entrenamiento sigue bajando, pero la de validación se estanca o sube. Es más probable cuantos más parámetros tiene el modelo en relación con los datos. Lo frenan la regularización (decaimiento de pesos, *dropout* o aumento de datos) y la parada temprana.

Parada temprana. Se evalúa en validación al final de cada época, se guarda la mejor y se para si no mejora durante un número de épocas seguidas, la paciencia [75].

Semilla. La inicialización, el orden de los lotes y el *dropout* dependen de números aleatorios. La semilla fija esa aleatoriedad para poder repetir un entrenamiento. Con semillas distintas, la misma receta puede dar resultados algo distintos.

5.8 Métricas de evaluación

Las métricas de texto se basan en la distancia de Levenshtein [76], que cuenta el número mínimo de errores entre la hipótesis y la referencia: sustituciones (S, palabras cambiadas), eliminaciones (D, palabras de la referencia que faltan en la hipótesis) e inserciones (I, palabras de más en la hipótesis). La tasa de error por palabra (WER, *Word Error Rate*) divide ese número entre las N palabras de la referencia:

$$\mathrm{WER} = \frac{S + D + I}{N}. \tag{5.29}$$

Por ejemplo, si la referencia es *i want some water* y el modelo escribe *i want the water please*, hay una sustitución (*the* en lugar de *some*) y una inserción (*please*), así que el WER es $2/4 = 50\,\%$. Como el número de inserciones no tiene límite, el WER puede pasar del 100 % si el modelo escribe mucho más de lo que había.

Sobre un conjunto de frases, el WER puede calcularse como la media de los WER de cada frase o sumando antes los errores y las palabras de todas. Lo habitual es lo segundo, el WER de corpus, que da a cada frase un peso proporcional a su longitud:

$$\mathrm{WER}_{\mathrm{corpus}} = \frac{\sum_i (S_i + D_i + I_i)}{\sum_i N_i}. \tag{5.30}$$

La tasa de error por carácter (CER, *Character Error Rate*) se calcula igual, pero con caracteres en lugar de palabras. Es menos severa, porque una palabra con una sola letra mal cuenta como un error de una letra y no de una palabra entera. ESPnet la calcula durante el entrenamiento sobre la salida de la cabeza CTC. Algunos trabajos dan, en lugar del error, el acierto, que es su complementario: un acierto por palabra del 95 % corresponde a un WER del 5 %.

Por último, la exactitud (*accuracy*, acc) que da ESPnet durante el entrenamiento es la proporción de tokens que acierta el decodificador cuando recibe como contexto los tokens reales (*teacher forcing*):

$$\mathrm{acc} = \frac{1}{\sum_i L_i} \sum_i \sum_{l=1}^{L_i} \mathbb{I}\left[\arg\max_{v \in \mathcal{V}} p(v \mid y_{i,<l}, \mathbf{X}_i) = y_{i,l} \right], \tag{5.31}$$

donde $\mathbb{I}[\cdot]$ vale 1 si se cumple la condición y 0 si no. Es rápida de calcular, porque no hace falta generar texto, y por eso se usa para seguir el entrenamiento y elegir épocas. Suele ser más optimista que el WER: con el contexto real, un error no afecta a los tokens siguientes, mientras que al generar texto un error al principio puede desviar el resto de la frase.

CAPÍTULO 6
Metodología

La metodología tiene dos fases, que corresponden a los dos bloques de experimentos de la sección 2.1. En la primera se busca el mejor modelo posible con un presupuesto de cómputo limitado, comparando lo que mejora cada cambio con lo que cuesta. En la segunda se toma ese modelo y se estudia cómo se comporta con el paso del tiempo y cuántos datos necesita. Antes se describen el sistema de partida y la forma de evaluarlo. Todos los entrenamientos se han hecho en Google Colab, con una sola GPU NVIDIA L4, usando ESPnet a través de su interfaz ESPnet-EZ [34], [35].

6.1 Adaptación de OWSM a la señal neuronal

El punto de partida es OWSM v3.1 en su versión base (sección 5.4). Para que trabaje con señal neuronal en lugar de audio se hacen estos cambios:

- El *frontend* acústico y el submuestreo convolucional se eliminan. En su lugar se pone un *frontend* nuevo (sección 5.3) que recibe las 512 características de cada ventana, tal y como se publican, y entrega vectores de dimensión 384. La codificación posicional se reutiliza de OWSM, porque no depende del tipo de entrada.
- El codificador mantiene la arquitectura E-Branchformer de OWSM v3.1 base, con 6 bloques y $d = 384$. Según el experimento, sus pesos son los preentrenados o aleatorios.
- El decodificador es el de OWSM v3.1 base, con su vocabulario de subpalabras y sus tokens especiales. Cada frase se le presenta como `<eng><asr><notimestamps>` seguido del texto, igual que una transcripción en inglés.
- La cabeza CTC trabaja sobre el mismo vocabulario y se entrena junto con el *frontend*.

El modelo se entrena con la pérdida conjunta de la ecuación (5.22), con $\lambda = 0{,}3$, y con InterCTC en la segunda capa del codificador, con $w = 0{,}3$ (ecuación (5.23)). El *frontend* y la cabeza CTC usan una tasa de aprendizaje de 10^{-3}, y el codificador, una 20 veces menor. El resto de la receta está en el **??**.

6.2 Evaluación

Como el test oficial no tiene transcripciones públicas, la validación oficial hace de conjunto de prueba (sección 4.2). Las cuatro sesiones sin validación solo aportan datos de entrenamiento. Antes de calcular el error, la referencia y la hipótesis se normalizan:

se pasan a minúsculas y se quitan los signos de puntuación, salvo los apóstrofos y los guiones.

Cada métrica tiene un papel distinto:

Para comparar modelos. El WER de corpus (ecuación (5.30)) en toda la validación, con la búsqueda en haz de la sección 5.5.3: haz de 5, peso de CTC de 0,3, detección automática del final de la frase y sin modelo de lenguaje externo. Es la cifra con la que se comparan los modelos. Se da también el WER de la decodificación CTC voraz, que muestra lo que consigue el codificador sin la ayuda del decodificador.

Para elegir la época. La acc de ESPnet (ecuación (5.31)). En la fase 1 se calcula sobre la validación y en la fase 2, sobre una reserva sacada del entrenamiento (sección 6.4).

Para medir la eficiencia. El número de parámetros, totales y entrenables, y el tiempo de entrenamiento en horas de GPU. En los *frontends* se miden además, sin entrenar, las operaciones por segundo de señal (GFLOPs) y la memoria máxima que ocupan en la GPU.

Para comprobar que un modelo lee de verdad la señal, se evalúa también con una señal falsa: a cada frase se le da la señal de otra. Si el error apenas cambia, el modelo no está usando la señal y se limita a escribir frases probables con el decodificador.

Hay dos limitaciones que afectan a la lectura de los resultados. En la fase 1, la época se elige con la misma validación con la que se evalúa, lo que da cifras algo optimistas, aunque igual de optimistas para todos los modelos; la fase 2 lo evita con la reserva. Además, por coste, cada configuración se entrena una sola vez y con una sola semilla, así que diferencias de uno o dos puntos de WER pueden deberse al azar.

6.3 Fase 1: búsqueda del mejor modelo

El objetivo de esta fase es encontrar el mejor modelo dentro de un coste razonable, es decir, ver qué cambios compensan lo que cuestan. Se parte de modelos sencillos y con pocos datos, y se van añadiendo, de uno en uno, elementos que usa la literatura.

6.3.1. Puesta en marcha

Las primeras pruebas se hicieron con unas diez sesiones, porque cada época tardaba unos 20 minutos en el equipo disponible, y con modelos sencillos: un *frontend* lineal y pocas capas del codificador sin congelar. Al principio el modelo no aprendía. El decodificador escribía casi la misma frase para cualquier entrada y el error por carácter rondaba el 99 %.

Para saber si el fallo estaba en el código o en el modelo, se hizo una prueba de memorización: entrenar con muy pocos datos y comprobar que el modelo era capaz de sobreajustarlos. Si un modelo no puede aprender de memoria unas pocas frases, hay un error en la cadena de datos o en el propio modelo; si puede, el problema está en la generalización. El modelo pasó la prueba.

El problema se resolvió al pasar a Colab, donde se cambiaron varias cosas a la vez: se pasó de unas diez sesiones a las 45, se descongelaron más capas del codificador y el *frontend* lineal se cambió por uno convolucional profundo. Como los cambios coincidieron, no se puede saber cuánto aportó cada uno, aunque lo que más parece haber influido es usar todas las sesiones.

6.3.2. Receta general y capa por sesión

A partir de ahí se fijó una receta general (**??**): como mucho 80 épocas, con parada temprana, quedándose con la época de mejor acc en validación. Cada experimento de la fase cambia una sola cosa respecto a esa receta.

Lo primero que se probó fue la capa de entrada por sesión (sección 5.2.2), que usan casi todos los sistemas del campo. En este caso no mejoró el resultado y, además, no sirve para días sin datos de entrenamiento, que son justo los que se estudian en la fase 2. Por eso no forma parte de la receta.

6.3.3. Codificador (OE2)

Con el decodificador de OWSM congelado, se comparan cuatro formas de tratar el codificador (sección 5.6): con los pesos preentrenados congelados, con los pesos preentrenados y LoRA, con los pesos preentrenados ajustados por completo y con pesos aleatorios, es decir, entrenado desde cero. El resto del sistema y la receta no cambian. Si lo que OWSM ha aprendido del audio sirve para la señal neuronal, las variantes preentrenadas deberían aprender antes o llegar más lejos que la aleatoria.

6.3.4. Decodificador (OE3)

Con el mejor codificador se estudia el decodificador. Primero se comprueba que el decodificador preentrenado aporta algo frente a una opción más sencilla: decodificar la salida CTC con un modelo de lenguaje de n-gramas, entrenado solo con las frases de entrenamiento, que estima la probabilidad de cada palabra a partir de las anteriores [77]. Después se comparan tres variantes:

- el decodificador preentrenado y congelado;
- el mismo con LoRA (sección 5.6.1), de rango 8, en las proyecciones de consultas y valores de la autoatención y de la atención cruzada de cada capa;
- un decodificador con la misma arquitectura entrenado desde cero.

La variante que se elige aquí es la que se usa en el resto del trabajo.

6.3.5. *Frontend* (OE1)

Por último, con el codificador y el decodificador ya fijados, se comparan los *frontends* de la tabla 6.1. Van de menos a más complejos, cada uno añade algo al anterior y casi todos están tomados de la literatura. Además del WER se mide el coste, porque el *frontend* decide cuántas ventanas procesa el codificador, que es la parte cara del modelo.

El *frontend* profundo empieza con una capa lineal de 512 a 512 con LayerNorm y GELU. Después vienen una o dos etapas, cada una con una convolución de núcleo 5 y paso 2, que reduce la secuencia a la mitad, y dos bloques convolucionales residuales de núcleo 5 con LayerNorm, GELU y *dropout* de 0,1. Termina con una capa lineal a 384 y la codificación posicional. Con una etapa, cada vector de salida ve 420 ms de señal. El submuestreo convolucional original de OWSM y una variante con una capa recurrente solo se midieron en coste, sin entrenarlos, porque eran más lentos.

Todos se entrenan con la misma receta y la misma semilla, durante 40 épocas y sin parada temprana, para que tengan el mismo presupuesto. Así la comparación es justa,

Tabla 6.1: *Frontends* comparados en la fase 1.

Frontend	Qué hace	Submuestreo
Lineal ×2	Apila 2 ventanas y aplica una capa lineal.	×2
Patching ×2	Agrupa 14 ventanas (280 ms) con paso 2 y aplica una capa lineal, como el sistema de referencia de Brain-to-Text '25 [30].	×2
Conv1d ×2	Dos convoluciones temporales de núcleo 3 con GELU, como la entrada de Whisper [36].	×2
Profundo ×2	Diseño propio (ver el texto), con una etapa.	×2
Profundo ×4	Diseño propio, con dos etapas.	×4
Patching ×4	El *frontend* del sistema de referencia de Brain-to-Text '25 tal cual.	×4

aunque las cifras no son el máximo que podría dar cada uno. El profundo ×2 es el mismo sistema de los apartados anteriores, de modo que sirve también de control: sus primeras 40 épocas deben reproducir las de aquel entrenamiento.

La combinación de codificador, decodificador y *frontend* que sale de esta fase es la receta final, la que se usa en la fase 2.

6.4 Fase 2: comportamiento en el tiempo y cantidad de datos

6.4.1. Variabilidad entre sesiones (OE4)

Antes de simular el uso real, se evalúa sesión a sesión, sin reentrenarlo, el modelo de la fase 1 con el decodificador congelado, que se entrenó con todas las sesiones. Así se ve cuánto cambia el error de un día a otro incluso cuando el modelo ha visto datos de todos los días.

6.4.2. Entrenamiento cronológico (OE4)

Después se simula cómo se usaría el sistema en la realidad: se graban sesiones, se entrena con ellas y se usa en los días siguientes. Para ello se entrena desde cero un modelo con las k primeras sesiones, en orden de fecha, para $k = 5, 10, \ldots, 40$; con las 45 ya no quedarían sesiones futuras. Entrenar un modelo para cada valor de k habría costado demasiado, así que se avanza de cinco en cinco. Todos usan la receta final de la fase 1.

Como aquí la validación oficial se usa solo para evaluar, cada modelo aparta como reserva 1 de cada 10 frases de entrenamiento de cada sesión. Con ella se elige la mejor época y se hace la parada temprana, con una paciencia de 6 épocas. Cada modelo da como mucho 40 pasadas por sus datos y como poco 4.000 pasos. Las épocas de ESPnet se fijan en 250 pasos, unas 8.000 frases, para todos los modelos, así que con pocas sesiones una época equivale a varias pasadas por los datos: con $k = 5$, unas siete.

Cada modelo se evalúa en toda la validación. De cada frase se guarda su relación con el entrenamiento: si su sesión estaba entre las de entrenamiento (sesión vista) o es posterior (sesión futura), cuántas sesiones y cuántos días han pasado desde la última sesión de entrenamiento, si la frase aparece tal cual en el entrenamiento, cuánto se parece a la frase de entrenamiento más parecida y cuántas de sus palabras aparecen en él. Con esto se comparan:

- el error en sesiones vistas y en sesiones futuras para cada k;

- el error en las sesiones futuras con el de un modelo que sí las vio: el profundo $\times 4$ de la fase 1, entrenado con todas las sesiones y evaluado sobre las mismas frases;
- el error en las cinco últimas sesiones (de la 41 a la 45, de enero a abril de 2025), que son futuras para todos los modelos y permiten compararlos sobre las mismas frases.

También se guardan las curvas de aprendizaje, para comprobar que cada modelo llega a su meseta y que un mal resultado no se debe a haber entrenado poco.

6.4.3. Análisis por palabras

Por último, con las hipótesis guardadas, se analiza el error por palabras: qué palabras acierta el modelo, si acierta más las que aparecen a menudo en el entrenamiento y qué pasa con las que no ha visto nunca. Junto con la relación de cada frase con el entrenamiento, esto permite separar el efecto del paso del tiempo del de encontrarse frases o palabras nuevas.

CAPÍTULO 7
Resultados y discusión

7.1 Organización de la presentación

Los resultados se presentan en el orden en que se obtuvieron, y no en el orden lógico del plan original. Ese orden es en sí mismo un resultado: el hallazgo del primer bloque invalidó la hipótesis de partida y obligó a reordenar los bloques posteriores. Presentarlos en el orden previsto daría la imagen falsa de un diseño cerrado desde el principio.

Salvo indicación expresa, las cifras corresponden a la tasa de error por carácter con decodificación CTC voraz, calculada de forma micro sobre la partición de validación completa (sección 6.2). Cuando se comparan objetivos de salida distintos, el eje común es la tasa de error por palabra, conforme a la sección 5.8.

Antes de los bloques conviene registrar la comprobación de capacidad. Entrenado sobre un subconjunto de 348 ensayos hasta el sobreajuste deliberado, el sistema alcanza una tasa de error por carácter de 0,052. La cadena completa es, por tanto, capaz de aprender la tarea: cuanto se observe después es un problema de generalización y no de conexión entre la señal y el modelo.

7.2 Bloques A y H: la aportación del preentrenamiento

7.2.1. Resultados

Tabla 7.1: Aportación del preentrenamiento. Tasa de error por carácter mínima sobre validación, época del mínimo, presupuesto y coste en unidades de cómputo.

Ejec.	Codificador	CER	Época	Épocas	Coste
Inicialización aleatoria, ajuste completo					
H1	Semilla 2	**0,297**	40	40	8,8
H2	Semilla 3	0,314	40	40	8,8
A1	Semilla 1	0,324	28	30	**6,6**
Pesos de OWSM					
v5	Ajuste completo	0,362	60	60	15,2
v3	Ajuste completo	0,381	37	38	8,9
D0	Congelado, presupuesto ampliado	0,440	70	70	14,6
A2	Congelado	0,658	29	30	6,3
H3	Ajuste completo, tasa cuadruplicada	0,689	39	40	8,7
Controles adicionales					
A4	Aleatorio, congelado	0,414	34	40	8,8
$A1_{lr}$	Aleatorio, tasa unificada	0,854*	—	30	6,5

* Ejecución divergente. Véase la sección 7.2.4.

Las tres semillas con inicialización aleatoria promedian 0,312 con una desviación típica de 0,014 y un rango de $[0{,}297,\ 0{,}324]$. La mejor ejecución con pesos de OWSM y receta comparable alcanza 0,381, situada a **5,1 desviaciones típicas** por encima de la media aleatoria. La diferencia excede con holgura tanto la dispersión entre semillas como el intervalo de confianza de la medida establecido en la **??**.

[FIGURA: curvas de CER en validación frente a época para A1, H1 y H2 (aleatorias, en un color), v3 y v5 (OWSM, en otro), y A2/D0 (congelado). Es la figura principal de la memoria: debe verse la banda de las tres semillas aleatorias enteramente por debajo de las preentrenadas.]

Figura 7.1: Evolución de la tasa de error por carácter según la inicialización del codificador.

7.2.2. Las dos objeciones previsibles, resueltas

El bloque H se diseñó para anticipar las dos objeciones evidentes a un resultado que contradice la literatura previa. Ambas quedan resueltas en contra de la objeción.

¿Es reproducible? Sí. Las tres semillas ocupan un intervalo de 2,7 puntos porcentuales, y la peor de ellas sigue siendo 5,7 puntos mejor que la mejor ejecución preentrenada de receta comparable. El efecto no es la fluctuación de una única ejecución afortunada.

¿Se le dio a OWSM su mejor oportunidad? Sí, y empeora. La hipótesis natural era que la tasa de aprendizaje reducida del codificador impidiese a los pesos preentrenados alejarse lo suficiente de su punto de partida, penalizando por construcción a la condición preentrenada. Al cuadruplicar esa tasa el resultado no mejora, sino que **se degrada de** 0,381 **a** 0,689. Lejos de estar infra-adaptados, los pesos de OWSM se destruyen en cuanto se les permite moverse con libertad, sin que el modelo recupere por ello el rendimiento de una inicialización aleatoria. La objeción queda refutada empíricamente.

7.2.3. El control que cierra el argumento

El resultado más contundente del bloque no es ninguna de las dos comparaciones anteriores, sino el control adicional A4: un codificador **inicializado aleatoriamente y congelado**, esto es, una proyección aleatoria fija sobre la que sólo se entrenan el adaptador y la cabeza CTC.

Ese control alcanza 0,414 en 40 épocas, frente a los 0,440 que obtiene el codificador de OWSM congelado tras 70 épocas, casi el doble de presupuesto. Dicho de otro modo: **como extractor de características fijo, un codificador aleatorio sin entrenar resulta al menos tan útil como uno preentrenado sobre 180.000 horas de habla**, y lo consigue con menos cómputo.

Este control transforma una conclusión débil en una fuerte. No se trata únicamente de que el preentrenamiento no aporte información aprovechable: es que las representaciones que induce no superan a una proyección aleatoria, lo que sitúa su contenido transferible a esta modalidad en un valor indistinguible de cero.

7.2.4. La tasa de aprendizaje diferenciada es un requisito de estabilidad

Tres ejecuciones independientes con tasa unificada entre codificador y componentes nuevos divergieron: la de inicialización aleatoria (0,854) y las dos de descongelado parcial (0,848 y 0,859). En todas ellas la pérdida de entrenamiento se estanca mientras la de validación asciende, patrón característico de divergencia y no de entrenamiento lento.

Que el fenómeno se reproduzca tanto con codificador aleatorio como con codificador preentrenado confirma lo anticipado en la **??**: la tasa diferenciada no cumple aquí la función de proteger unos pesos preentrenados, sino la de estabilizar el acoplamiento entre un adaptador que aprende deprisa y un codificador profundo cuyas representaciones aún no están organizadas.

7.2.5. Contraste con la literatura previa

El resultado contradice a Fiedler *et al.* [51], que aplicando el mismo control sobre wav2vec 2.0 y los datos del participante T12 obtuvieron una ventaja de más de veinte puntos porcentuales *a favor* de los pesos preentrenados.

Con la evidencia ahora disponible, dos de las cuatro explicaciones que cabía plantear quedan descartadas. La restricción impuesta por la tasa de aprendizaje se descarta por el resultado de H3. Y la hipótesis de que un adaptador demasiado potente estuviese absorbiendo la función del codificador preentrenado se descarta por el bloque C (sección 7.7):

al reducir la capacidad del adaptador el sistema empeora de forma monótona, sin que la ventaja del preentrenamiento reaparezca en ningún punto.

Quedan en pie dos explicaciones, ambas relativas a *qué* se preentrena y no a *cómo* se adapta:

H1. Naturaleza del preentrenamiento. wav2vec 2.0 se preentrena de forma autosupervisada sobre forma de onda cruda mediante un extractor convolucional, mientras que OWSM lo hace con supervisión débil sobre banco de filtros mel. Un codificador construido sobre una representación espectral estaría especializado en una estructura de la que la señal neuronal carece por completo, mientras que las representaciones autosupervisadas de bajo nivel serían más agnósticas a la modalidad. La hipótesis predice que la propiedad determinante no es haber visto habla, sino el nivel de abstracción al que se aprendió a representarla.

H2. Diferencias del conjunto de datos. El horizonte de 20 meses del corpus empleado aquí, frente a los cuatro meses del utilizado por ellos, impone una carga de no estacionariedad muy superior, que podría dominar sobre cualquier ventaja de inicialización.

Ninguna de las dos es contrastable con los recursos de este trabajo, y ambas se recogen como línea de continuación en el capítulo 8.

7.3 Bloque D: la curva de conservación

Tabla 7.2: Rendimiento en función de la cantidad de codificador preentrenado que se adapta. La pendiente final es la del descenso de la tasa de error en la última época del presupuesto.

Ejec.	Condición	CER	Épocas	Pendiente final
A2	OWSM congelado	0,658	30	−0,022
D0	OWSM congelado, ampliado	0,440	70	−0,001
D2	OWSM, 2 capas superiores	0,521	30	−0,008
D3	OWSM, 4 capas superiores	0,493	30	−0,008
D1	OWSM congelado con LoRA	0,403	30	−0,004
v3	OWSM, ajuste completo	0,381	38	−0,004
A1	Aleatorio, ajuste completo	**0,324**	30	−0,001

La curva **no es monótona**, y su interpretación exige atender a la columna de pendientes. Las dos condiciones de descongelado parcial conservan al final del presupuesto una pendiente de $-0{,}008$ por época, un orden de magnitud superior a la de las condiciones convergidas: no han alcanzado su meseta y sus cifras no representan su rendimiento asintótico. El mismo argumento explica el salto del codificador congelado, de 0,658 a 0,440, al más que duplicar su presupuesto.

Descontando las condiciones no convergidas, el orden que sí puede afirmarse es que **el ajuste completo supera a la adaptación de rango bajo, esta supera a la congelación, y la inicialización aleatoria supera a todas**. Resulta destacable que la adaptación de rango bajo sobre codificador congelado (0,403) mejore en 3,7 puntos a la congelación simple pese a disponer de menos de la mitad de presupuesto (0,440 con 70 épocas), lo que confirma que su beneficio procede de la restricción del subespacio de adaptación y no de la capacidad añadida, conforme al argumento de la sección 5.6.1.

Lo que la curva no muestra en ningún punto es un óptimo intermedio: no existe una cantidad de preentrenamiento conservado que resulte preferible a prescindir de él por completo.

7.4 Bloque F: el tamaño de la arquitectura

Establecido que los pesos no aportan, queda por determinar si lo hace el tamaño de la arquitectura que los albergaba.

Tabla 7.3: Efecto de la profundidad del codificador con inicialización aleatoria.

Bloques	CER	WER voraz	Épocas	Minutos
6 (arquitectura de OWSM)	0,324	0,571	30	232
3	0,334	0,569	40	272
2	0,353	0,602	40	259

Reducir el codificador a la mitad cuesta un punto porcentual de tasa de error por carácter y deja la tasa de error por palabra prácticamente inalterada. Reducirlo a un tercio cuesta tres puntos. La degradación es, por tanto, muy suave, lo que indica que **tampoco el tamaño de la arquitectura de OWSM está justificado por la tarea**: el problema admite codificadores sensiblemente menores sin pérdida apreciable.

Una precisión honesta sobre esta tabla: las ejecuciones reducidas se entrenaron 40 épocas frente a las 30 de la completa, de modo que el ahorro no se materializa en la columna de tiempo total. El ahorro real está en el coste por época, y una comparación limpia de eficiencia exigiría igualar también el presupuesto.

7.5 Bloque B: granularidad de la unidad de salida

7.5.1. Resultados

Tabla 7.4: Comparación de unidades de salida con decodificación voraz. La columna de error por palabra es el único eje comparable entre filas.

Unidad	Clases	CER	WER	Épocas
Palabra	Cerrado	**0,222**	**0,282**	40
Subpalabra de OWSM	50.002	0,247	0,323	20
Subpalabra propia	256	0,251	0,424	40
Carácter	29	0,267	0,535	40
Fonema	—	No produce texto		40

El resultado es monótono y **contrario a la predicción formulada en los capítulo 4 y ??**: cuanto más gruesa es la unidad de salida, mejor es la transcripción final. La diferencia entre el mejor y el peor objetivo que produce texto asciende a 25 puntos porcentuales de tasa de error por palabra, muy por encima de cualquier otro efecto medido en el trabajo, incluida la contribución del preentrenamiento.

En particular, el vocabulario de subpalabras de OWSM, que esta memoria había descartado *a priori* por escasez de ejemplos por clase, resulta ser el segundo mejor objetivo, y lo consigue con la mitad de presupuesto que los demás y sin haber alcanzado su meseta:

su mínimo se produce en la penúltima época con una pendiente todavía de $-0{,}007$ por época.

7.5.2. Por qué falló la predicción

El razonamiento original era que repartir poco más de ocho mil frases entre 50.002 clases dejaría a la inmensa mayoría sin ejemplos, y que un vocabulario reducido sería por tanto preferible. El razonamiento es correcto en los hechos y erróneo en la consecuencia, por dos motivos que la literatura de reconocimiento del habla tenía documentados y que no se tuvieron en cuenta.

El primero es que **en CTC las clases no observadas no penalizan**. Una clase que nunca aparece en el entrenamiento simplemente no se predice: no compite por masa de probabilidad ni degrada a las demás. El coste de un vocabulario grande es de memoria y de tiempo de cálculo de la función *softmax*, no de calidad.

El segundo, y determinante, es el efecto de la unidad sobre la **longitud de la secuencia objetivo**. Frente a los caracteres, las subpalabras producen secuencias de salida más cortas y reducen así la dificultad de modelar la dependencia entre salidas, algo especialmente importante en modelos basados en CTC por su hipótesis de independencia condicional [78]. El fenómeno es conocido: con unidades grandes como la palabra, las limitaciones de CTC se sortean en parte, mientras que con unidades pequeñas como el carácter se vuelven prominentes [79]. En reconocimiento del habla convencional, sustituir el objetivo de carácter por subpalabras produce reducciones relativas de tasa de error del orden del 13 % sin diccionario ni modelo de lenguaje externo [80].

A ello se añade una circunstancia específica de este montaje. Con ventanas de 20 ms y submuestreo por dos, la secuencia de entrada queda a una trama cada 40 ms. CTC exige que la entrada sea más larga que la etiqueta, y con objetivo de carácter esa holgura se estrecha considerablemente. Es un problema documentado al reducir la resolución temporal, y su solución habitual consiste precisamente en adoptar subpalabras en lugar de caracteres [81].

La conclusión metodológica es que **el argumento de los ejemplos por clase, aplicado sin considerar la longitud de la secuencia objetivo, conduce a la decisión equivocada**. Es un error de razonamiento a priori que sólo el experimento podía corregir, y constituye por sí mismo un argumento a favor de contrastar empíricamente las decisiones de diseño en lugar de deducirlas.

7.5.3. Matices y limitaciones del hallazgo

Tres precisiones impiden leer la tabla como una recomendación simple de emplear siempre la unidad más gruesa.

El objetivo de **palabra opera sobre vocabulario cerrado** y es, por construcción, incapaz de emitir una palabra no vista en entrenamiento. Dado que la partición de validación incorpora deliberadamente corpus ausentes del entrenamiento (sección 4.2), su ventaja está inflada respecto a lo que ofrecería en uso real, donde el vocabulario abierto es un requisito. Se trata de una limitación estructural y no de una cuestión de volumen de datos: la literatura señala precisamente la ausencia del problema de palabras fuera de vocabulario como la ventaja de caracteres y subpalabras frente a palabras [80].

El objetivo de **fonema no produce texto ortográfico**, de modo que no puede situarse en el eje común sin un decodificador con léxico. Su ausencia de la comparación es una limitación reconocida y resulta especialmente lamentable, ya que es la unidad que emplea

la práctica totalidad de la literatura del campo y la que el argumento neurocientífico señalaba como natural.

Por último, la comparación se realiza con **decodificación voraz y sin modelo de lenguaje externo**. Es previsible que la fusión con un modelo de lenguaje reduzca la ventaja de las unidades gruesas, puesto que buena parte de lo que aportan es precisamente la estructura lingüística implícita que el modelo de lenguaje suministra por otra vía.

7.5.4. Respuesta a PI2

La corteza motora del habla puede leerse a granularidades muy distintas, pero **la unidad que minimiza el error del texto final no es la más próxima a la articulación sino la más próxima al lenguaje**. El factor determinante no es la plausibilidad neurocientífica de la unidad, sino la longitud de la secuencia que induce y la cantidad de estructura lingüística que incorpora implícitamente, compensando la hipótesis de independencia condicional de CTC.

7.6 Bloque E: generalización a sesiones no vistas

Tabla 7.5: Rendimiento sobre las cinco sesiones excluidas del entrenamiento, según el tratamiento de la capa de sesión.

Tratamiento	CER	WER voraz	Época del mínimo
Sin capa de sesión	**0,593**	0,848	33
Matriz en la identidad	0,747	0,984	10
Media de las matrices entrenadas	0,752	0,970	12

El resultado es inequívoco: ante un día de registro no visto, **el mejor tratamiento de la capa de sesión es no tenerla**. Prescindir de ella mejora en más de 15 puntos porcentuales sobre cualquiera de las dos estrategias de sustitución, y ni la identidad ni el promedio de las matrices entrenadas ofrecen ventaja apreciable la una sobre la otra.

La columna de la época del mínimo revela el mecanismo. Las dos configuraciones con capa de sesión alcanzan su mejor rendimiento sobre días no vistos en las épocas 10 y 12, y a partir de ahí se degradan pese a que su rendimiento sobre días vistos continúa mejorando. La capa de sesión no está aprendiendo una corrección generalizable de la deriva del implante, sino **memorizando cada día concreto**, y esa memorización resulta progresivamente perjudicial fuera de los días memorizados.

Contrastado con el resto del trabajo, el compromiso es severo. Con todos los días disponibles y capa de sesión, el sistema alcanza 0,324; sobre días no vistos, la mejor configuración se sitúa en 0,593. La degradación de casi 27 puntos porcentuales es, con diferencia, la mayor medida en todo el trabajo, y sitúa la no estacionariedad, y no la elección de codificador, como el problema dominante del despliegue real. Es también la conclusión de mayor relevancia clínica, por corresponder al escenario del primer día tras el implante o de la primera sesión tras un periodo sin uso.

Los aproximadamente doce millones de parámetros que cuesta la capa de sesión compran, en definitiva, rendimiento dentro de la distribución a cambio de capacidad de generalización fuera de ella.

7.7 Bloque C: capacidad del módulo adaptador

Tabla 7.6: Comparación de variantes de adaptador. Las tres comparten factor de submuestreo dos.

Variante	CER	WER voraz	Minutos
Profunda	**0,324**	0,571	232
Convolucional 1D	0,399	0,643	244
Lineal	0,475	0,710	185

La advertencia formulada en la **??** sobre una posible confusión entre arquitectura del adaptador y resolución temporal **no llegó a materializarse**: las tres variantes se ejecutaron con factor de submuestreo dos, de modo que la comparación es limpia.

El adaptador profundo justifica su coste con claridad, mejorando en 7,5 puntos porcentuales a la variante convolucional y en 15,1 a la lineal, con un tiempo por época comparable al de la primera. La separación explícita entre mezcla espacial de electrodos y convolución temporal, argumentada en la **??**, se traduce en una diferencia sustancial de rendimiento.

Este bloque cumple además una función no prevista en el plan original, la de contrastar la hipótesis de que un adaptador excesivamente potente estuviese absorbiendo la función del codificador preentrenado. La predicción de esa hipótesis era que la ventaja del preentrenamiento reapareciese al reducir la capacidad del adaptador. No ocurre: al debilitarlo, el sistema empeora de forma monótona y en ningún punto la condición preentrenada resulta preferible.

7.8 Decodificación con modelo de lenguaje

Los resultados de esta sección proceden de una configuración anterior a la fijación de la receta definitiva y **deben considerarse preliminares**. Se reportan porque el efecto que documentan es cualitativo y previsiblemente robusto al cambio de codificador, pero sus cifras absolutas habrán de recalcularse.

Tabla 7.7: Resultados preliminares de decodificación con modelo de lenguaje de n-gramas por fusión superficial.

Condición	WER
Fusión superficial con bigrama	0,475
Límite *oracle* de la lista n-mejor acústica	0,503

El resultado cualitativamente relevante es que **la fusión superficial supera al límite *oracle***. Conforme al argumento de la **??**, ningún método de reordenamiento puede superar ese límite, definido como el error que se obtendría eligiendo siempre la mejor hipótesis presente en la lista. Que el sistema quede por debajo demuestra que la transcripción correcta **no figuraba** en la lista de hipótesis puramente acústicas y que el modelo de lenguaje la alcanzó interviniendo durante la expansión del haz.

La consecuencia para PI3 es directa: una parte del conocimiento lingüístico del sistema completo no reside en el codificador neuronal, y el modelo de lenguaje no es una etapa de posprocesado sino un componente que altera el espacio de búsqueda. El hallazgo se refuerza con el del bloque B: si las unidades de salida gruesas mejoran el texto final en 25 puntos porcentuales, es porque incorporan implícitamente parte de esa misma

estructura lingüística dentro del propio codificador. Ambos resultados apuntan en la misma dirección: **en este sistema, la reconstrucción lingüística pesa más que la lectura de la señal**.

7.9 Comparación con el estado del arte

Tabla 7.8: Situación del sistema propuesto frente a la referencia comparable sobre el mismo conjunto de datos.

Sistema	Codificador	CER	WER
Khanday *et al.* [46]	Conformer desde cero	0,238	—
Este trabajo, objetivo de carácter	E-Branchformer aleatorio	0,267	0,535
Este trabajo, objetivo de palabra	E-Branchformer aleatorio	0,222	0,282

Con objetivo de carácter y decodificación voraz, que es la condición directamente comparable, el sistema alcanza 0,267 frente al 0,238 que reportan Khanday *et al.* [46] sobre el mismo conjunto, la misma unidad y las mismas condiciones de decodificación. La diferencia de unos tres puntos porcentuales **no es atribuible al preentrenamiento**, puesto que ninguno de los dos sistemas lo emplea en esa configuración, y apunta a la arquitectura del codificador, al aumento de datos o a la receta de optimización.

Es un margen estrecho para un trabajo que no perseguía competir, y sitúa el sistema en el mismo orden de magnitud que la literatura reciente sobre este conjunto. Cambiando el objetivo a palabra la tasa de error por carácter desciende a 0,222, si bien con la salvedad del vocabulario cerrado ya señalada.

La distancia con los sistemas que encabezan la clasificación oficial sigue siendo, no obstante, considerable, y responde a los factores identificados en el capítulo 3: conjuntos de decenas de modelos, modelos de lenguaje de gran tamaño para la fusión de hipótesis y presupuestos de cómputo de otro orden de magnitud.

7.10 Discusión

7.10.1. Síntesis por pregunta de investigación

PI1, transferencia. El preentrenamiento de OWSM sobre habla **no transfiere** a la decodificación de actividad neuronal intracortical. El resultado es reproducible entre semillas, resiste el intento de conceder a la condición preentrenada un ajuste más favorable, y se refuerza con el control decisivo de que un codificador aleatorio congelado iguala o supera al codificador preentrenado congelado con la mitad de presupuesto. Tampoco el tamaño de la arquitectura resulta justificado por la tarea.

PI2, granularidad. La corteza puede leerse a granularidades muy distintas, y la que minimiza el error del texto final es la más gruesa, no la más próxima a la articulación. El factor determinante es la longitud de la secuencia objetivo y la estructura lingüística que la unidad incorpora implícitamente. Este eje produce el mayor efecto medido en todo el trabajo, muy por encima del preentrenamiento.

PI3, localización del conocimiento lingüístico. Una parte sustancial del conocimiento lingüístico reside fuera del codificador. El modelo de lenguaje externo guía la búsqueda en lugar de limitarse a reordenar hipótesis, y las unidades de salida gruesas mejoran el resultado precisamente por internalizar parte de esa misma función.

7.10.2. Una lectura conjunta

Los tres resultados convergen en una conclusión que no formaba parte de las hipótesis iniciales: **en este problema, y con este volumen de datos, lo que determina el rendimiento no es la calidad de la representación acústica heredada sino la cantidad de estructura lingüística que el sistema logra incorporar**, ya sea a través de la unidad de salida, del modelo de lenguaje externo o de ambos. El preentrenamiento sobre habla aporta lo primero y no lo segundo, lo que ofrece una explicación coherente de por qué su contribución resulta nula.

Frente a ello, el factor limitante del despliegue no es ninguno de los tres ejes estudiados, sino la no estacionariedad: la degradación de 27 puntos porcentuales ante un día de registro no visto supera a cualquier otro efecto medido, y la solución estándar del campo, la capa específica de sesión, resulta ser en ese escenario parte del problema y no de la solución.

7.10.3. Amenazas a la validez

Un solo participante. Todas las conclusiones son específicas de T15, de la localización de sus implantes y del inglés como lengua objetivo.

Un solo modelo preentrenado. El resultado se refiere a OWSM. No autoriza a concluir que ningún modelo de habla transfiera, y la evidencia previa sobre wav2vec 2.0 apunta a lo contrario.

Presupuestos desiguales entre algunas condiciones. Varias comparaciones enfrentan ejecuciones de 30 y 40 épocas. Cuando esto favorece a la condición que se declara peor, la conclusión resulta conservadora; en el bloque F, en cambio, impide una comparación limpia de eficiencia.

Condiciones no convergidas. Las dos condiciones de descongelado parcial conservan pendiente apreciable al final de su presupuesto y sus cifras no son asintóticas.

Evaluación sobre validación. Al no distribuirse la partición de prueba con etiquetas, todas las cifras corresponden a validación, sobre la que se han tomado decisiones de diseño, con el consiguiente riesgo de sobreajuste que las cifras no capturan.

Modelo de lenguaje sin reevaluar. Las cifras de la sección 7.8 proceden de una configuración anterior y no se han recalculado sobre el sistema final.

CAPÍTULO 8
Limitaciones y trabajo futuro

Este capítulo reúne las limitaciones del estudio y las líneas de continuación que se derivan de sus resultados. Se separa de las amenazas a la validez recogidas en la sección 7.10, que se refieren a la interpretación de cifras concretas, para tratar aquí las restricciones estructurales del trabajo y lo que cabría hacer a continuación.

8.1 Limitaciones del estudio

8.1.1. Alcance de la evidencia

La limitación de mayor peso es que **todas las conclusiones proceden de un único participante**. El conjunto de datos empleado registra a una sola persona, con una localización concreta de implantes y con el inglés como lengua objetivo. Que el preentrenamiento de OWSM no transfiera en T15 no implica que no lo haga en otro sujeto, con otra cobertura cortical o en otro idioma, aunque la magnitud y la consistencia del efecto medido hacen poco plausible que se invierta por completo.

Análogamente, el estudio evalúa **un único modelo preentrenado**. La conclusión sobre transferencia se refiere a OWSM v3.1 y no puede generalizarse al conjunto de los modelos de habla, especialmente teniendo en cuenta que la evidencia previa disponible sobre wav2vec 2.0 apunta en dirección contraria [51].

8.1.2. Restricciones del entorno de ejecución

El trabajo se desarrolla sobre una única GPU de gama media en un servicio en la nube, con un presupuesto de cómputo acotado. Esta restricción excluye por construcción las técnicas que sostienen los primeros puestos de la clasificación oficial, esto es, los conjuntos de decenas de modelos y el uso de modelos de lenguaje de gran tamaño para la fusión de hipótesis [43], y condiciona además el modelo de lenguaje empleado, entrenado sobre el propio corpus por no caber en memoria el oficial de la competición [82].

Una consecuencia metodológica de esta restricción es que las ablaciones se ejecutan con presupuesto igualado y no hasta convergencia, de modo que sus cifras no representan el rendimiento máximo de cada configuración. Dos condiciones concretas, las de descongelado parcial del codificador, conservaban pendiente apreciable al agotarse su presupuesto y su comparación queda por tanto abierta.

8.1.3. Elementos del plan que no se completaron

Tres piezas quedaron fuera del alcance efectivo del trabajo y conviene declararlas.

La **evaluación con modelo de lenguaje no se recalculó sobre la configuración final**. Las cifras que sustentan la tercera pregunta de investigación proceden de una configuración anterior, de modo que su valor es cualitativo y no cuantitativo. El argumento sobre el límite *oracle* se mantiene, por ser estructural, pero la magnitud de la contribución del modelo de lenguaje al sistema definitivo no está medida.

El **objetivo de fonema quedó fuera de la comparación de granularidades**, al no producir texto ortográfico y no disponerse de un decodificador con léxico que permitiera situarlo en el eje común de tasa de error por palabra. Es una ausencia notable, dado que el fonema es la unidad que emplea la práctica totalidad de la literatura del campo y la que el argumento neurocientífico señalaba como natural.

Por último, la **evaluación sobre la partición de prueba oficial** no es posible con los datos públicos, que no incluyen sus etiquetas. Todas las cifras reportadas corresponden a la partición de validación, sobre la que además se tomaron decisiones de diseño a lo largo del trabajo.

8.2 Líneas de trabajo futuro

8.2.1. Discriminar qué tipo de preentrenamiento transfiere

La línea más directa que abre este trabajo procede de su contradicción con la literatura previa. Que wav2vec 2.0 transfiera y OWSM no, sobre tareas emparentadas, sugiere que la propiedad determinante no es haber sido entrenado sobre habla sino **el nivel de abstracción al que se aprendió a representarla**. Un modelo autosupervisado sobre forma de onda cruda aprende estructura temporal de bajo nivel potencialmente agnóstica a la modalidad; uno supervisado sobre banco de filtros mel aprende estructura espectral de la que la señal neuronal carece.

Contrastar esta hipótesis exigiría un estudio comparativo de varios modelos preentrenados bajo un mismo esquema de adaptación y sobre un mismo conjunto de datos, variando de forma controlada el objetivo de preentrenamiento (autosupervisado frente a supervisado) y la representación de entrada (forma de onda frente a espectrograma). Sería, en esencia, la generalización natural del bloque A de este trabajo, y respondería a una pregunta que la comunidad todavía no tiene resuelta.

Una variante de menor coste consistiría en evaluar POWSM, un modelo de la misma familia preentrenado con objetivo fonémico, que permitiría comprobar si la transferencia reaparece cuando la unidad del preentrenamiento coincide con la unidad natural de la corteza motora del habla.

8.2.2. Atacar la no estacionariedad como problema central

El resultado con mayor implicación práctica de este trabajo es que la degradación ante un día de registro no visto, de casi 27 puntos porcentuales, supera con creces a cualquier otro efecto medido, y que la solución estándar del campo resulta contraproducente en ese escenario por memorizar días concretos en lugar de aprender una corrección generalizable.

Esto sitúa la adaptación entre sesiones, y no la elección de codificador, como el problema dominante. Las líneas naturales son la regularización explícita de las matrices de sesión hacia una solución compartida, el aprendizaje de una función que **prediga** la

transformación de un día nuevo a partir de una pequeña muestra sin etiquetar de ese día, y la exploración de proyecciones jerárquicas a nivel de mes y de día como las que emplean los sistemas más recientes [45]. Cuantificar cuántos minutos de calibración se necesitan para recuperar el rendimiento nominal en un día nuevo tendría además valor clínico directo.

8.2.3. Cerrar el eje de granularidad

El bloque de granularidad produjo el mayor efecto del trabajo y quedó incompleto en dos puntos. El primero es la incorporación del objetivo de fonema al eje común, que requiere un decodificador con léxico y permitiría contrastar por fin el argumento neurocientífico con datos propios. El segundo es determinar si la ventaja de las unidades gruesas se mantiene al añadir un modelo de lenguaje externo, o si por el contrario se desvanece porque ambos mecanismos aportan la misma estructura lingüística por vías distintas. Esta segunda pregunta es la más informativa de las dos, porque separa la contribución interna de la externa.

Cabe además explorar granularidades intermedias no evaluadas aquí, en particular los difonos, que capturan explícitamente la coarticulación y con los que el equipo ganador de la edición anterior del *benchmark* obtuvo su mejora principal [29], así como esquemas jerárquicos que apliquen pérdidas de granularidad creciente a capas sucesivas del codificador [78].

8.2.4. Explotar la eficiencia del codificador reducido

Que reducir el codificador a la mitad cueste un punto porcentual abre una línea de eficiencia con implicaciones para el despliegue en tiempo real, donde la latencia es un requisito y no una métrica secundaria. Un estudio sistemático de la frontera entre coste y rendimiento, incluyendo anchura además de profundidad y midiendo latencia de inferencia en lugar de tiempo de entrenamiento, permitiría determinar el modelo mínimo viable para uso conversacional.

8.2.5. Reproducir el hallazgo en otros participantes

Por último, y en cuanto se disponga de conjuntos públicos con varios participantes implantados, la comprobación pendiente más importante es si el resultado central se reproduce fuera de T15. El campo avanza hacia modelos entrenados conjuntamente sobre múltiples sujetos e incluso especies [43], y ese marco permitiría además contrastar la hipótesis alternativa de que lo que no transfiere no es el preentrenamiento sobre habla en general, sino el preentrenamiento sobre habla *ajena*, frente a un preentrenamiento sobre actividad neuronal de otros sujetos.

CAPÍTULO 9
Conclusiones

Este trabajo se planteó como un estudio de viabilidad: determinar si un modelo de reconocimiento del habla preentrenado a gran escala puede reutilizarse como codificador de actividad neuronal intracortical para decodificar texto, y bajo qué condiciones. La respuesta a esa pregunta resultó ser negativa, y el trabajo se convirtió en el proceso en algo distinto de lo previsto: en una caracterización de qué factores sí determinan el rendimiento en esta tarea.

9.1 Respuesta a las preguntas de investigación

9.1.1. PI1: el preentrenamiento de OWSM no transfiere

La primera pregunta era si el preentrenamiento de un modelo de habla aporta ventaja medible frente a la misma arquitectura inicializada al azar. **No la aporta, y la penaliza**. Con receta idéntica y presupuesto no favorable a la condición ganadora, un codificador inicializado aleatoriamente alcanza una tasa de error por carácter de 0,312 de media sobre tres semillas, frente a 0,381 del mismo codificador inicializado con los pesos de OWSM, y lo hace convergiendo antes y con la mitad de coste.

El hallazgo resiste las dos objeciones que cabía anticipar. Es **reproducible**: las tres semillas ocupan un intervalo de 2,7 puntos y la peor sigue muy por delante de la condición preentrenada. Y no es un artefacto del ajuste: al conceder a los pesos de OWSM una tasa de aprendizaje cuadruplicada, el rendimiento no mejora sino que se degrada hasta 0,689.

El control que cierra el argumento es, sin embargo, otro. Un codificador **aleatorio y congelado**, es decir, una proyección fija sin entrenar, alcanza 0,414 frente a los 0,440 del codificador de OWSM congelado con casi el doble de presupuesto. Como extractor de características fijo, ciento ochenta mil horas de habla no superan al azar. El contenido transferible del preentrenamiento a esta modalidad es, por tanto, indistinguible de cero.

La curva completa de conservación no muestra en ningún punto un óptimo intermedio: no existe una cantidad de OWSM que convenga conservar. Y el bloque de profundidad añade que tampoco el tamaño de su arquitectura está justificado por la tarea, ya que reducir el codificador a la mitad cuesta un único punto porcentual.

9.1.2. PI2: la unidad óptima es la más lingüística, no la más articulatoria

La segunda pregunta era a qué granularidad conviene leer la corteza motora del habla. El resultado es monótono y contrario a la predicción con que se partía: **cuanto más gruesa es la unidad de salida, mejor es el texto final**. Sobre el eje común de tasa de error por

palabra, el objetivo de palabra alcanza 0,282, el de subpalabra preentrenada 0,323, el de subpalabra propia 0,424 y el de carácter 0,535.

Los 25 puntos porcentuales que separan los extremos constituyen el mayor efecto medido en todo el trabajo, muy por encima de la contribución del preentrenamiento. Y el vocabulario de 50.002 clases, que esta memoria había descartado por razonamiento a priori sobre escasez de ejemplos por clase, resulta ser el segundo mejor objetivo, alcanzándolo con la mitad de presupuesto y sin haber convergido.

La explicación no está en la neurociencia sino en las propiedades del objetivo de entrenamiento: las clases no observadas no penalizan bajo CTC, y las unidades gruesas producen secuencias objetivo más cortas que mitigan la hipótesis de independencia condicional. Es un resultado conocido en reconocimiento del habla que el razonamiento previo de este trabajo pasó por alto.

9.1.3. PI3: el conocimiento lingüístico reside fuera del codificador

La tercera pregunta era dónde reside el conocimiento lingüístico del sistema completo. La evidencia disponible, preliminar en sus cifras pero estructural en su argumento, indica que **una parte sustancial reside fuera del codificador neuronal**. La fusión superficial con un modelo de lenguaje de n-gramas alcanza una tasa de error por palabra de 0,475, inferior al límite *oracle* de 0,503 de la lista de hipótesis puramente acústicas. Puesto que ningún método de reordenamiento puede superar ese límite, queda demostrado que la transcripción correcta no figuraba entre las hipótesis del modelo acústico y que el modelo de lenguaje la alcanzó guiando la búsqueda.

Este resultado converge con el anterior. Si las unidades de salida gruesas mejoran el texto final en 25 puntos, es porque internalizan dentro del codificador parte de la misma estructura lingüística que el modelo de lenguaje aporta desde fuera. Ambos hallazgos señalan que, en este sistema, **la reconstrucción lingüística pesa más que la lectura de la señal**.

9.2 Aportaciones

Las aportaciones de este trabajo pueden resumirse en cinco puntos:

1. **Un resultado negativo medido con control explícito.** La ausencia de transferencia desde OWSM se establece frente a un control de inicialización aleatoria, con réplica entre semillas, con la objeción del ajuste desfavorable refutada y con un control de proyección aleatoria congelada que descarta la explicación alternativa. La diferencia entre no encontrar transferencia y no haberla medido es precisamente lo que separa este trabajo de los que asumen el beneficio del preentrenamiento sin comprobarlo.
2. **Una comparación sistemática de granularidades de salida** sobre un mismo modelo, un mismo presupuesto y un mismo eje de evaluación, que no se había realizado sobre este conjunto de datos y cuyo resultado contradice la práctica dominante del campo.
3. **Evidencia de que la capa de sesión, solución estándar frente a la no estacionariedad, es contraproducente ante días no vistos**, por memorizar días concretos en lugar de aprender una corrección generalizable.
4. **Una cadena de experimentación reproducible** sobre datos públicos, herramientas de código abierto y una única GPU de gama media, con el presupuesto de cómputo documentado experimento a experimento.

5. **Un sistema competitivo con la literatura reciente sobre el mismo conjunto**: 0,267 de tasa de error por carácter frente al 0,238 de la referencia comparable, en condiciones idénticas de unidad y decodificación.

9.3 Lecciones metodológicas

Al margen de los resultados, el trabajo deja tres lecciones que condicionaron su desarrollo y que se consideran transferibles.

La primera es que **los valores por omisión de las herramientas gobiernan decisiones que no son suyas**. El criterio de parada temprana de la herramienta empleada monitoriza la pérdida y no la métrica de la tarea, lo que interrumpió un entrenamiento que seguía mejorando y conservó modelos peores durante buena parte del trabajo. Ningún valor por omisión que gobierne la selección de modelo debería darse por bueno sin comprobarlo.

La segunda es que **el razonamiento a priori sobre decisiones de diseño es poco fiable incluso cuando sus premisas son ciertas**. La predicción de que un vocabulario de 50.002 clases sería inviable partía de un hecho correcto, la escasez de ejemplos por clase, y llegaba a la conclusión opuesta a la verdadera por no considerar un segundo factor. El experimento no fue una confirmación de lo previsto sino su refutación, que es exactamente para lo que sirve.

La tercera es que **una diferencia sólo es un resultado si excede la resolución de la medida**. Con doscientos ensayos evaluados el intervalo de confianza ronda los tres a cinco puntos porcentuales, de modo que buena parte de las mejoras observadas en las primeras iteraciones del trabajo estaban por debajo del umbral de detección y no significaban nada.

9.4 Valoración final

El trabajo no ha demostrado lo que se propuso demostrar. Se planteó verificar que un modelo de habla preentrenado podía reutilizarse para decodificar actividad cortical, y ha establecido con bastante solidez que este modelo concreto no puede, o más precisamente que su preentrenamiento no aporta nada que no aporte igual una inicialización al azar.

Ese resultado tiene valor por tres razones. Está **medido y no supuesto**, lo que en un campo donde el preentrenamiento se adopta habitualmente sin control no es trivial. **Contradice a la literatura previa de forma informativa**, acotando la pregunta abierta de qué tipo de preentrenamiento produce representaciones reutilizables fuera de su modalidad, que es una pregunta más interesante que la que este trabajo se hizo originalmente. Y tiene **consecuencias prácticas**: si los pesos no aportan y el tamaño tampoco, el problema admite modelos sensiblemente menores y más baratos que los que la intuición del campo sugeriría.

Los dos hallazgos colaterales resultan, además, de mayor alcance práctico que el central. Que la unidad de salida produzca un efecto de 25 puntos porcentuales, cuatro veces mayor que el del preentrenamiento, señala dónde conviene invertir el esfuerzo de diseño. Y que la degradación ante un día de registro no visto alcance los 27 puntos, con la solución estándar del campo agravándola, identifica la no estacionariedad como el obstáculo real que separa estos sistemas de un uso clínico cotidiano.

Puesto que el objetivo declarado era un estudio de viabilidad y no un sistema de competición, un resultado que delimita con precisión qué no funciona, por qué, y qué sí importa en su lugar, cumple ese objetivo tanto como lo habría cumplido el resultado contrario.

Bibliografía

[1] A. B. Silva, K. T. Littlejohn, J. R. Liu, D. A. Moses y E. F. Chang, «The Speech Neuroprosthesis,» *Nature Reviews Neuroscience*, vol. 25, n.º 7, págs. 473-492, 2024. DOI: 10.1038/s41583-024-00819-9

[2] L.-X. Lin et al., «Stroke-Associated Dysarthria,» *Frontiers in Neurology*, vol. 16, 2025, **artno** 1629640. DOI: 10.3389/fneur.2025.1629640

[3] D. Beukelman, S. Fager y A. Nordness, «Communication Support for People with ALS,» *Neurology Research International*, vol. 2011, 2011, **artno** 714693. DOI: 10.1155/2011/714693

[4] L. Xu et al., «Global Variation in Prevalence and Incidence of Amyotrophic Lateral Sclerosis: A Systematic Review and Meta-Analysis,» *Journal of Neurology*, vol. 267, n.º 4, págs. 944-953, 2020. DOI: 10.1007/s00415-019-09652-y

[5] Servimedia. «Los casos de ELA aumentarán un 40 % en Europa en los próximos 25 años.» Datos de la Sociedad Española de Neurología (SEN), Servicio de Información sobre Discapacidad (SID-INICO), Universidad de Salamanca, visitado 7 de nov. de 2026. dirección: https://sid-inico.usal.es/noticias/los-casos-de-ela-aumentaran-un-40-en-europa-en-los-proximos-25-anos/

[6] GBD 2021 Stroke Risk Factor Collaborators, «Global, Regional, and National Burden of Stroke and Its Risk Factors, 1990–2021: A Systematic Analysis for the Global Burden of Disease Study 2021,» *The Lancet Neurology*, vol. 23, n.º 10, págs. 973-1003, 2024. DOI: 10.1016/S1474-4422(24)00369-7

[7] T. Sánchez. «Cerca de 90.000 nuevos casos de ictus en España cada año: "Ha disminuido la mortalidad pero tenemos más personas con secuelas".» Datos de la Sociedad Española de Neurología (SEN), Vozpópuli, visitado 7 de nov. de 2026. dirección: https://www.vozpopuli.com/sanitatem/cerca-de-90000-nuevos-casos-de-ictus-en-espana-cada-ano-ha-disminuido-la-mortalidad-pero-tenemos-mas-personas-con-secuelas.html

[8] M. Summaka et al., «Neuroanatomical Regions Associated with Non-Progressive Dysarthria Post-Stroke: A Systematic Review,» *BMC Neurology*, vol. 22, pág. 353, 2022. DOI: 10.1186/s12883-022-02877-x

[9] E. Smith y M. Delargy, «Locked-In Syndrome,» *BMJ*, vol. 330, n.º 7488, págs. 406-409, 2005. DOI: 10.1136/bmj.330.7488.406

[10] M.-A. Bruno, J. L. Bernheim, D. Ledoux, F. Pellas, A. Demertzi y S. Laureys, «A Survey on Self-Assessed Well-Being in a Cohort of Chronic Locked-In Syndrome Patients: Happy Majority, Miserable Minority,» *BMJ Open*, vol. 1, n.º 1, e000039, 2011. DOI: 10.1136/bmjopen-2010-000039

[11] G. C. Vanderheiden, «A Journey through Early Augmentative Communication and Computer Access,» *Journal of Rehabilitation Research and Development*, vol. 39, n.º 6, suplemento, págs. 39-53, 2002.

[12] R. Spataro, M. Ciriacono, C. Manno y V. La Bella, «The Eye-Tracking Computer Device for Communication in Amyotrophic Lateral Sclerosis,» *Acta Neurologica Scandinavica*, vol. 130, n.º 1, págs. 40-45, 2014. DOI: 10.1111/ane.12214

[13] M. J. Vansteensel et al., «Fully Implanted Brain–Computer Interface in a Locked-In Patient with ALS,» *New England Journal of Medicine*, vol. 375, n.º 21, págs. 2060-2066, 2016. DOI: 10.1056/NEJMoa1608085

[14] T. Schultz, M. Wand, T. Hueber, D. J. Krusienski, C. Herff y J. S. Brumberg, «Biosignal-Based Spoken Communication: A Survey,» *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 25, n.º 12, págs. 2257-2271, 2017. DOI: 10.1109/TASLP.2017.2752365

[15] L. A. Farwell y E. Donchin, «Talking off the Top of Your Head: Toward a Mental Prosthesis Utilizing Event-Related Brain Potentials,» *Electroencephalography and Clinical Neurophysiology*, vol. 70, n.º 6, págs. 510-523, 1988. DOI: 10.1016/0013-4694(88)90149-6

[16] N. Birbaumer et al., «A Spelling Device for the Paralysed,» *Nature*, vol. 398, n.º 6725, págs. 297-298, 1999. DOI: 10.1038/18581

[17] D. A. Moses et al., «Neuroprosthesis for Decoding Speech in a Paralyzed Person with Anarthria,» *New England Journal of Medicine*, vol. 385, n.º 3, págs. 217-227, 2021. DOI: 10.1056/NEJMoa2027540

[18] S. L. Metzger et al., «A High-Performance Neuroprosthesis for Speech Decoding and Avatar Control,» *Nature*, vol. 620, n.º 7976, págs. 1037-1046, 2023. DOI: 10.1038/s41586-023-06443-4

[19] M. Angrick et al., «Real-Time Synthesis of Imagined Speech Processes from Minimally Invasive Recordings of Neural Activity,» *Communications Biology*, vol. 4, pág. 1055, 2021. DOI: 10.1038/s42003-021-02578-0

[20] L. R. Hochberg et al., «Neuronal Ensemble Control of Prosthetic Devices by a Human with Tetraplegia,» *Nature*, vol. 442, n.º 7099, págs. 164-171, 2006. DOI: 10.1038/nature04970

[21] C. Pandarinath et al., «High Performance Communication by People with Paralysis Using an Intracortical Brain-Computer Interface,» *eLife*, vol. 6, e18554, 2017. DOI: 10.7554/eLife.18554

[22] F. R. Willett, D. T. Avansino, L. R. Hochberg, J. M. Henderson y K. V. Shenoy, «High-Performance Brain-to-Text Communication via Handwriting,» *Nature*, vol. 593, n.º 7858, págs. 249-254, 2021. DOI: 10.1038/s41586-021-03506-2

[23] F. R. Willett et al., «A High-Performance Speech Neuroprosthesis,» *Nature*, vol. 620, n.º 7976, págs. 1031-1036, 2023. DOI: 10.1038/s41586-023-06377-x

[24] N. S. Card et al., «An Accurate and Rapidly Calibrating Speech Neuroprosthesis,» *New England Journal of Medicine*, vol. 391, n.º 7, págs. 609-618, 2024. DOI: 10.1056/NEJMoa2314132

[25] M. Wairagkar et al., «An Instantaneous Voice-Synthesis Neuroprosthesis,» *Nature*, vol. 644, n.º 8075, págs. 145-152, 2025. DOI: 10.1038/s41586-025-09127-3

[26] Instituto Nacional de Estadística. «Encuesta de Discapacidad, Autonomía Personal y Situaciones de Dependencia (EDAD). Año 2020.» Nota de prensa, visitado 7 de nov. de 2026. dirección: https://www.ine.es/prensa/edad_2020_p.pdf

[27] A. Londral, A. Pinto, S. Pinto, L. Azevedo y M. De Carvalho, «Quality of Life in Amyotrophic Lateral Sclerosis Patients and Caregivers: Impact of Assistive Communication from Early Stages,» *Muscle & Nerve*, vol. 52, n.º 6, págs. 933-941, 2015.

[28] Jefatura del Estado. «Ley 3/2024, de 30 de octubre, para mejorar la calidad de vida de personas con Esclerosis Lateral Amiotrófica y otras enfermedades o procesos de alta complejidad y curso irreversible.» BOE núm. 263, BOE-A-2024-22438, Boletín Oficial del Estado, visitado 7 de nov. de 2026. dirección: https://www.boe.es/eli/es/l/2024/10/30/3

[29] F. R. Willett et al., *Brain-to-Text Benchmark '24: Lessons Learned*, 2024. DOI: 10.48550/arXiv.2412.17227 arXiv: 2412.17227 [cs.CL].

[30] N. S. Card, F. R. Willett, S. D. Stavisky y D. M. Brandman, *Brain-to-Text Benchmark '25*, Competición y conjunto de datos públicos, Datos depositados en Dryad, https://doi.org/10.5061/dryad.dncjsxm85, 2025. visitado 4 de sep. de 2026. dirección: https://www.kaggle.com/competitions/brain-to-text-25

[31] T. J. Oxley et al., «Motor Neuroprosthesis Implanted with Neurointerventional Surgery Improves Capacity for Activities of Daily Living Tasks in Severe Paralysis: First In-Human Experience,» *Journal of NeuroInterventional Surgery*, vol. 13, n.º 2, págs. 102-108, 2021. DOI: 10.1136/neurintsurg-2020-016862

[32] Y. Peng et al., «Reproducing Whisper-Style Training Using an Open-Source Toolkit and Publicly Available Data,» en *IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2023. DOI: 10.1109/ASRU57964.2023.10389676

[33] Y. Peng et al., «OWSM v3.1: Better and Faster Open Whisper-Style Speech Models Based on E-Branchformer,» en *Proc. Interspeech 2024*, 2024, págs. 352-356. DOI: 10.21437/Interspeech.2024-1194

[34] S. Watanabe et al., «ESPnet: End-to-End Speech Processing Toolkit,» en *Proc. Interspeech 2018*, 2018, págs. 2207-2211. DOI: 10.21437/Interspeech.2018-1456

[35] M. Someki et al., «ESPnet-EZ: Python-Only ESPnet for Easy Fine-Tuning and Integration,» en *IEEE Spoken Language Technology Workshop (SLT)*, 2024. DOI: 10.1109/SLT61566.2024.10832299

[36] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey e I. Sutskever, «Robust Speech Recognition via Large-Scale Weak Supervision,» en *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ép. Proceedings of Machine Learning Research, vol. 202, PMLR, 2023, págs. 28 492-28 518. visitado 4 de sep. de 2026. dirección: https://proceedings.mlr.press/v202/radford23a.html

[37] C. Fan et al., «Plug-and-Play Stability for Intracortical Brain-Computer Interfaces: A One-Year Demonstration of Seamless Brain-to-Text Communication,» en *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. visitado 7 de nov. de 2026. dirección: https://proceedings.neurips.cc/paper_files/paper/2023/hash/83a14a36de4502bac5b580db36e81858-Abstract.html

[38] E. J. Hu et al., «LoRA: Low-Rank Adaptation of Large Language Models,» en *International Conference on Learning Representations (ICLR)*, 2022. visitado 4 de sep. de 2026. dirección: https://openreview.net/forum?id=nZeVKeeFYf9

[39] E. Feghhi, J. Hu, N. Hadidi y J. C. Kao, *LightBeam: An Accurate and Memory-Efficient CTC Decoder for Speech Neuroprostheses*, 2026. DOI: 10.48550/arXiv.2603.14002 arXiv: 2603.14002 [eess.AS].

[40] H. Jo, Y. Yang, J. Han, Y. Duan, H. Xiong y W. H. Lee, *Are EEG-to-Text Models Working?* 2024. DOI: 10.48550/arXiv.2405.06459 arXiv: 2405.06459.

[41] M. Verwoert et al., «Dataset of Speech Production in Intracranial Electroencephalography,» *Scientific Data*, vol. 9, 2022, **artno** 434. DOI: 10.1038/s41597-022-01542-9

[42] J. Kohler et al., «Synthesizing Speech from Intracranial Depth Electrodes Using an Encoder-Decoder Framework,» *Neurons, Behavior, Data Analysis, and Theory*, vol. 6, 2022. DOI: 10.51628/001c.57524

[43] Y. Zhang et al., «A Cross-Species Neural Foundation Model for End-to-End Speech Decoding,» en *International Conference on Learning Representations (ICLR)*, 2026. arXiv: 2511.21740.

[44] D. Sussillo, S. D. Stavisky, J. C. Kao, S. I. Ryu y K. V. Shenoy, «Making Brain–Machine Interfaces Robust to Future Neural Variability,» *Nature Communications*, vol. 7, 2016, **artno** 13749. DOI: 10.1038/ncomms13749

[45] T. Boccato, M. Olak y M. Ferrante, *BrainWhisperer: Leveraging Large-Scale ASR Models for Neural Speech Decoding*, 2026. DOI: 10.48550/arXiv.2603.13321 arXiv: 2603.13321 [eess.AS].

[46] O. M. Khanday, J. A. Gonzalez-Lopez, M. Ouellet, A. Galdón y G. Olivares Granados, «End-to-End Intracortical Speech Decoding from Neural Activity,» en *Proc. Odyssey 2026: The Speaker and Language Recognition Workshop*, 2026. DOI: 10.21437/Odyssey.2026-24

[47] D. Jurafsky y J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*, 3.ª ed. 2026, Borrador en línea, versión del 19 de agosto de 2026. visitado 8 de nov. de 2026. dirección: https://web.stanford.edu/~jurafsky/slp3/

[48] A. Baevski, H. Zhou, A. Mohamed y M. Auli, «wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,» en *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, págs. 12 449-12 460.

[49] M. Kim, Z. Piao, J. Lee y H.-G. Kang, *BrainTalker: Low-Resource Brain-to-Speech Synthesis with Transfer Learning Using Wav2Vec 2.0*, 2023. DOI: 10.48550/arXiv.2312.13600 arXiv: 2312.13600 [eess.AS].

[50] B. A. Yuan y J. G. Makin, *Improving Speech Decoding from ECoG with Self-Supervised Pretraining*, 2024. DOI: 10.48550/arXiv.2405.18639 arXiv: 2405.18639 [eess.AS].

[51] T. Fiedler et al., *Teaching Wav2Vec2 the Language of the Brain*, 2025. DOI: 10.48550/arXiv.2501.09459 arXiv: 2501.09459 [cs.LG].

[52] I. Vales Cortina et al., «UGR-MINDVOICE: A Multimodal EEG-Audio Dataset for Overt and Covert Iberian Spanish Speech Production,» *Computer Speech & Language*, vol. 100, 2026, **artno** 101964. DOI: 10.1016/j.csl.2026.101964

[53] J. J. Godfrey, E. C. Holliman y J. McDaniel, «SWITCHBOARD: Telephone Speech Corpus for Research and Development,» en *Proc. IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, vol. 1, 1992, págs. 517-520. DOI: 10.1109/ICASSP.1992.225858

[54] IEEE, «IEEE Recommended Practice for Speech Quality Measurements,» *IEEE Transactions on Audio and Electroacoustics*, vol. 17, n.º 3, págs. 225-246, 1969. DOI: 10.1109/TAU.1969.1162058

[55] D. Hendrycks y K. Gimpel, *Gaussian Error Linear Units (GELUs)*, 2016. arXiv: 1606.08415.

[56] J. L. Ba, J. R. Kiros y G. E. Hinton, *Layer Normalization*, 2016. arXiv: 1607.06450.

[57] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever y R. Salakhutdinov, «Dropout: A Simple Way to Prevent Neural Networks from Overfitting,» *Journal of Machine Learning Research*, vol. 15, n.º 56, págs. 1929-1958, 2014.

[58] K. He, X. Zhang, S. Ren y J. Sun, «Deep Residual Learning for Image Recognition,» en *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, págs. 770-778. DOI: 10.1109/CVPR.2016.90

[59] A. Vaswani et al., «Attention Is All You Need,» en *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017, págs. 5998-6008.

[60] Y. Peng et al., «OWSM v4: Improving Open Whisper-Style Speech Models via Data Scaling and Cleaning,» en *Proc. Interspeech 2025*, 2025. arXiv: 2506.00338.

[61] T. Kudo y J. Richardson, «SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing,» en *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2018, págs. 66-71. DOI: 10.18653/v1/D18-2012

[62] A. Gulati et al., «Conformer: Convolution-Augmented Transformer for Speech Recognition,» en *Proc. Interspeech 2020*, 2020, págs. 5036-5040. DOI: 10.21437/Interspeech.2020-3015

[63] Y. Peng, S. Dalmia, I. Lane y S. Watanabe, «Branchformer: Parallel MLP-Attention Architectures to Capture Local and Global Context for Speech Recognition and Understanding,» en *Proc. 39th International Conference on Machine Learning (ICML)*, ép. PMLR, vol. 162, 2022, págs. 17 627-17 643.

[64] K. Kim et al., «E-Branchformer: Branchformer with Enhanced Merging for Speech Recognition,» en *IEEE Spoken Language Technology Workshop (SLT)*, 2023, págs. 84-91. DOI: 10.1109/SLT54892.2023.10022656

[65] R. J. Williams y D. Zipser, «A Learning Algorithm for Continually Running Fully Recurrent Neural Networks,» *Neural Computation*, vol. 1, n.º 2, págs. 270-280, 1989. DOI: 10.1162/neco.1989.1.2.270

[66] A. Graves, S. Fernández, F. Gomez y J. Schmidhuber, «Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks,» en *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 2006, págs. 369-376. DOI: 10.1145/1143844.1143891

[67] S. Kim, T. Hori y S. Watanabe, «Joint CTC-Attention Based End-to-End Speech Recognition Using Multi-Task Learning,» en *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, págs. 4835-4839. DOI: 10.1109/ICASSP.2017.7953075

[68] S. Watanabe, T. Hori, S. Kim, J. R. Hershey y T. Hayashi, «Hybrid CTC/Attention Architecture for End-to-End Speech Recognition,» *IEEE Journal of Selected Topics in Signal Processing*, vol. 11, n.º 8, págs. 1240-1253, 2017. DOI: 10.1109/JSTSP.2017.2763455

[69] J. Lee y S. Watanabe, «Intermediate Loss Regularization for CTC-Based Speech Recognition,» en *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021. arXiv: 2102.03216.

[70] J. Yosinski, J. Clune, Y. Bengio y H. Lipson, «How Transferable Are Features in Deep Neural Networks?» En *Advances in Neural Information Processing Systems 27 (NeurIPS 2014)*, 2014, págs. 3320-3328.

[71] X. Glorot e Y. Bengio, «Understanding the Difficulty of Training Deep Feedforward Neural Networks,» en *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, ép. Proceedings of Machine Learning Research, vol. 9, 2010, págs. 249-256.

[72] D. P. Kingma y J. Ba, «Adam: A Method for Stochastic Optimization,» en *International Conference on Learning Representations (ICLR)*, 2015. arXiv: 1412.6980.

[73] I. Loshchilov y F. Hutter, «Decoupled Weight Decay Regularization,» en *International Conference on Learning Representations (ICLR)*, 2019. arXiv: 1711.05101.

[74] R. Pascanu, T. Mikolov e Y. Bengio, «On the Difficulty of Training Recurrent Neural Networks,» en *Proceedings of the 30th International Conference on Machine Learning (ICML)*, ép. Proceedings of Machine Learning Research, vol. 28, 2013, págs. 1310-1318.

[75] L. Prechelt, «Early Stopping — But When?» En *Neural Networks: Tricks of the Trade*, ép. Lecture Notes in Computer Science, G. B. Orr y K.-R. Müller, eds., vol. 1524, Springer, 1998, págs. 55-69. DOI: 10.1007/3-540-49430-8_3

[76] V. I. Levenshtein, «Binary Codes Capable of Correcting Deletions, Insertions, and Reversals,» *Soviet Physics Doklady*, vol. 10, n.º 8, págs. 707-710, 1966.

[77] K. Heafield, «KenLM: Faster and Smaller Language Model Queries,» en *Proceedings of the Sixth Workshop on Statistical Machine Translation*, Association for Computational Linguistics, 2011, págs. 187-197. visitado 4 de sep. de 2026. dirección: https://aclanthology.org/W11-2123/

[78] Y. Higuchi, K. Karube, T. Ogawa y T. Kobayashi, «Hierarchical Conditional End-to-End ASR with CTC and Multi-Granular Subword Units,» en *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, págs. 7797-7801. DOI: 10.1109/ICASSP43922.2022.9746580

[79] A. Das, J. Li, R. Zhao e Y. Gong, «Advancing Connectionist Temporal Classification with Attention Modeling,» en *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, págs. 4769-4773. DOI: 10.1109/ICASSP.2018.8461558

[80] A. Zeyer, K. Irie, R. Schlüter y H. Ney, «Improved Training of End-to-End Attention Models for Speech Recognition,» en *Proc. Interspeech 2018*, 2018, págs. 7-11. DOI: 10.21437/Interspeech.2018-1616

[81] G. Yang, Z. Ma, Z. Zheng, Y. Song, Z. Niu y X. Chen, *Fast-HuBERT: An Efficient Training Framework for Self-Supervised Speech Representation Learning*, 2023. DOI: 10.48550/arXiv.2309.13860 arXiv: 2309.13860 [eess.AS].

[82] J. Cheng. «Solución de séptimo puesto: Mamba, GRU y KenLM en el Brain-to-Text '25.» Solución publicada de la competición Brain-to-Text '25, visitado 4 de sep. de 2026.

APÉNDICE A

Código del pipeline de entrenamiento

APÉNDICE B

Análisis exploratorio adicional

APÉNDICE C

Relación del trabajo con los Objetivos de Desarrollo Sostenible de la Agenda 2030

Objetivos de Desarrollo Sostenible	Alto	Medio	Bajo	No procede
ODS 1. Fin de la pobreza.				
ODS 2. Hambre cero.				
ODS 3. Salud y bienestar.				
ODS 4. Educación de calidad.				
ODS 5. Igualdad de género.				
ODS 6. Agua limpia y saneamiento.				
ODS 7. Energía asequible y no contaminante.				
ODS 8. Trabajo decente y crecimiento económico.				
ODS 9. Industria, innovación e infraestructuras.				
ODS 10. Reducción de las desigualdades.				
ODS 11. Ciudades y comunidades sostenibles.				
ODS 12. Producción y consumo responsables.				
ODS 13. Acción por el clima.				
ODS 14. Vida submarina.				
ODS 15. Vida de ecosistemas terrestres.				
ODS 16. Paz, justicia e instituciones sólidas.				
ODS 17. Alianzas para lograr objetivos.				

Tabla C.1: Grado de relación del trabajo con los Objetivos de Desarrollo Sostenible.

Descripción de la alineación del TFG con los ODS con un grado de relación más alto

[Pendiente de redactar.]